Table 1: End-to-end execution time (s, median per iteration) and speedup split by pass type. When present, the OctTree row includes ColorOctree passes; a separate ColorOctree row reports only those passes. ADT fields = non-recursive fields annotated with `@BENCH adt_fields`; SoA bufs = 1+ non-recursive field slots across all constructors (recursive children are stored in the tag buffer). Speedup >1× means SoA is faster; **bold** marks >1.1×.

Program	**ADT**	**SoA**	**Fold passes**			**Map passes**		
	fields	bufs	AoS (s)	SoA (s)	Speedup	AoS (s)	SoA (s)	Speedup
OctTree	16	9	0.4012	0.3527	**1.14**×	0.3045	0.3484	0.87×
Compiler	9	8	0.1488	0.0656	**2.27**×	0.1596	0.1805	0.88×
DBQuery	15	13	0.1403	0.0941	**1.49**×	0.1373	0.1682	0.82×
DecisionTree	7	6	0.5336	0.6464	0.83×	–	–	–
DomTree	15	14	0.1371	0.0628	**2.18**×	0.0587	0.0821	0.72×
KDTree	17	16	0.1801	0.1413	**1.27**×	–	–	–
LinearListReduction	11	11	0.0306	0.0052	**5.92**×	–	–	–
List	2	2	0.1297	0.1115	**1.16**×	0.2533	0.3391	0.75×
MonoTree	3	2	0.0178	0.0224	0.79×	0.1006	0.1086	0.93×
ObjectGraph	5	4	0.1143	0.2314	0.49×	0.1407	0.1605	0.88×
PiecewiseFunctions	8	7	0.1457	0.0998	**1.46**×	0.2395	0.2642	0.91×
TernaryTree	5	3	0.0272	0.0248	1.10×	0.1028	0.0916	**1.12**×
Trie	10	9	0.0746	0.0452	**1.65**×	0.1486	0.1675	0.89×
ColorOctree	25	18	0.0893	0.0811	**1.10**×	–	–	–

Table 2: PAPI speedup summary across all passes. For each program and counter, speedup is AoS/SoA using summed per-pass median counters. >1× means SoA has fewer events.

Program	**CYC**	**INS**	**L1D**	**L1I**	**L2D**	**L2I**	**LLC**
OctTree	1.03×	0.75×	**1.36**×	0.49×	**1.46**×	0.65×	**3.80**×
Compiler	**1.49**×	0.52×	**4.11**×	0.62×	**16.86**×	0.42×	**7.34**×
DBQuery	1.08×	0.47×	1.09×	0.34×	**7.83**×	0.27×	**5.79**×
DecisionTree	0.82×	0.69×	0.79×	0.50×	0.43×	0.49×	0.93×
DomTree	**1.48**×	0.57×	**2.24**×	0.91×	**21.01**×	0.23×	**10.33**×
KDTree	**1.24**×	0.77×	**1.51**×	0.84×	0.33×	0.76×	**4.87**×
LinearListReduction	**5.40**×	0.90×	**32.21**×	**2.94**×	**46.31**×	**3.87**×	**10.28**×
List	0.81×	0.80×	**3.06**×	0.47×	0.13×	0.93×	**1.47**×
MonoTree	0.99×	0.85×	0.41×	0.51×	0.28×	0.87×	**1.23**×
ObjectGraph	0.94×	0.75×	**2.80**×	0.59×	0.43×	0.53×	**3.15**×
PiecewiseFunctions	1.07×	0.60×	**1.96**×	1.00×	**1.47**×	0.35×	**4.61**×
TernaryTree	**1.18**×	0.79×	**1.51**×	0.55×	**1.63**×	**1.79**×	**1.34**×
Trie	1.05×	0.56×	**3.33**×	0.65×	**3.04**×	0.21×	**5.81**×
Geomean	**1.22**×	0.68×	**2.12**×	0.67×	**1.85**×	0.60×	**3.58**×

Table 3: PAPI counter totals across all passes (AoS/SoA). Each entry is the sum of per-pass median counter values.

Program	CYC (A/S)	INS (A/S)	L1D (A/S)	L1I (A/S)	L2D (A/S)	L2I (A/S)	LLC (A/S)
OctTree	2.7e+09/**2.6e+09**	**8.2e+09**/1.1e+10	3.8e+07/**2.8e+07**	**4.0e+05**/8.2e+05	8.3e+05/**5.7e+05**	**1.4e+04**/2.1e+04	6.5e+07/**1.7e+07**
Compiler	8.8e+08/**5.9e+08**	**9.9e+08**/1.9e+09	1.3e+07/**3.3e+06**	**2.7e+05**/4.4e+05	1.5e+06/**9.2e+04**	**1.3e+04**/3.1e+04	5.1e+07/**6.9e+06**
DBQuery	9.5e+08/**8.8e+08**	**1.1e+09**/2.2e+09	4.0e+06/**3.6e+06**	**1.6e+05**/4.7e+05	8.4e+05/**1.1e+05**	**8.8e+03**/3.2e+04	2.1e+07/**3.5e+06**
DecisionTree	**2.7e+09**/3.3e+09	**6.3e+09**/9.0e+09	**7.9e+07**/1.0e+08	**1.1e+04**/2.2e+04	**5.9e+05**/1.4e+06	**1.0e+04**/2.1e+04	**1.6e+08**/1.7e+08
DomTree	8.2e+08/**5.5e+08**	**1.2e+09**/2.2e+09	1.2e+07/**5.1e+06**	**1.1e+05**/1.2e+05	1.1e+06/**5.4e+04**	**7.4e+03**/3.2e+04	5.7e+07/**5.6e+06**
KDTree	8.7e+08/**7.0e+08**	**3.1e+09**/4.0e+09	1.5e+07/**9.9e+06**	**1.6e+03**/1.9e+03	**3.6e+04**/1.1e+05	**1.3e+03**/1.7e+03	4.4e+07/**9.1e+06**
LinearListReduction	1.4e+08/**2.6e+07**	**9.0e+07**/1.0e+08	1.5e+06/**4.7e+04**	7.9e+02/**2.7e+02**	3.3e+05/**7.1e+03**	6.4e+02/**1.6e+02**	1.2e+07/**1.2e+06**
List	**1.2e+09**/1.5e+09	**4.5e+09**/5.6e+09	9.2e+06/**3.0e+06**	**2.6e+05**/5.5e+05	**2.6e+04**/2.0e+05	**9.8e+03**/1.1e+04	4.9e+07/**3.3e+07**
MonoTree	**2.5e+08**/2.6e+08	**9.1e+08**/1.1e+09	**2.2e+05**/5.4e+05	**3.4e+04**/6.7e+04	**3.8e+03**/1.3e+04	**2.0e+03**/2.3e+03	2.2e+06/**1.8e+06**
ObjectGraph	**8.1e+08**/8.7e+08	**3.0e+09**/4.0e+09	9.8e+06/**3.5e+06**	**1.5e+05**/2.5e+05	**3.6e+04**/8.4e+04	**1.2e+04**/2.3e+04	2.2e+07/**7.1e+06**
PiecewiseFunctions	1.2e+09/**1.2e+09**	**2.8e+09**/4.7e+09	1.4e+07/**7.2e+06**	**4.2e+05**/4.2e+05	1.8e+05/**1.2e+05**	**1.5e+04**/4.2e+04	4.4e+07/**9.5e+06**
TernaryTree	4.2e+08/**3.5e+08**	**1.2e+09**/1.5e+09	2.0e+06/**1.3e+06**	**6.8e+04**/1.2e+05	3.3e+04/**2.0e+04**	3.7e+03/**2.1e+03**	3.2e+06/**2.4e+06**
Trie	5.9e+08/**5.7e+08**	**1.2e+09**/2.2e+09	9.9e+06/**3.0e+06**	**1.8e+05**/2.7e+05	1.9e+05/**6.3e+04**	**7.3e+03**/3.5e+04	2.6e+07/**4.4e+06**

Table 4: Mutable vs immutable cursor comparison (times only). Times are median per iteration (s). **Bold** marks the fastest time across all four variants.

Program	Am	Ai	Sm	Si
Compiler	0.3085	1.109	**0.2462**	*OOM*
DBQuery	0.2776	0.3412	**0.2623**	1.114
DecisionTree	**0.5336**	1.029	0.6464	2.754
DomTree	0.1958	0.3441	**0.1448**	0.5270
KDTree	0.1801	0.2616	**0.1413**	0.7114
LinearListReduction	0.0306	0.1329	**0.0052**	*OOM*
List	**0.3830**	*OOM*	0.4506	*OOM*
MonoTree	**0.1184**	0.1833	0.1310	0.2825
ObjectGraph	**0.2550**	0.5034	0.3919	0.7123
OctTree_sumMass	0.0415	0.0436	**0.0306**	*OOM*
OctTree_sumEnergy	0.0451	0.1159	**0.0352**	*OOM*
OctTree_countActive	0.0376	0.0472	**0.0302**	*OOM*
OctTree_countParticles	0.0360	0.0564	**0.0291**	*OOM*
OctTree_barnesHutPotential	**0.0432**	0.0594	0.0445	*OOM*
OctTree_fmmPotential	0.1084	**0.0986**	0.1021	*OOM*
OctTree_scaleEnergy	**0.1502**	0.1535	0.1761	*OOM*
OctTree_clearFlags	0.1543	**0.1508**	0.1724	*OOM*
PiecewiseFunctions	0.3851	0.5816	**0.3640**	2.176
TernaryTree	0.1300	0.1627	**0.1164**	0.6182
Trie	0.2232	0.4307	**0.2127**	1.016
ColorOctree	0.0893	0.1273	**0.0811**	*OOM*

Table 5: Mutable vs immutable cursor comparison (speedups). Shown: AoS-mut/SoA-mut, AoS-imm/AoS-mut, AoS-imm/SoA-mut, AoS-imm/SoA-imm. >1× means the denominator is faster.

Program	Am/Sm	Ai/Am	Ai/Sm	Ai/Si
Compiler	**1.25×**	**3.60×**	**4.51×**	–
DBQuery	1.06×	**1.23×**	**1.30×**	0.31×
DecisionTree	0.83×	**1.93×**	**1.59×**	0.37×
DomTree	**1.35×**	**1.76×**	**2.38×**	0.65×
KDTree	**1.27×**	**1.45×**	**1.85×**	0.37×
LinearListReduction	**5.92×**	**4.34×**	**25.74×**	–
List	0.85×	–	–	–
MonoTree	0.90×	**1.55×**	**1.40×**	0.65×
ObjectGraph	0.65×	**1.97×**	**1.28×**	0.71×
OctTree_sumMass	**1.36×**	1.05×	**1.43×**	–
OctTree_sumEnergy	**1.28×**	**2.57×**	**3.30×**	–
OctTree_countActive	**1.25×**	**1.25×**	**1.56×**	–
OctTree_countParticles	**1.23×**	**1.57×**	**1.94×**	–
OctTree_barnesHutPotential	0.97×	**1.37×**	**1.34×**	–
OctTree_fmmPotential	1.06×	0.91×	0.97×	–
OctTree_scaleEnergy	0.85×	1.02×	0.87×	–
OctTree_clearFlags	0.90×	0.98×	0.88×	–
PiecewiseFunctions	1.06×	**1.51×**	**1.60×**	0.27×
TernaryTree	**1.12×**	**1.25×**	**1.40×**	0.26×
Trie	1.05×	**1.93×**	**2.03×**	0.42×
ColorOctree	**1.10×**	**1.42×**	**1.57×**	–

Table 6: Runtime comparison of Gibbon (AoS/SoA), GHC. Times are total median per iteration (s). **Bold** marks the fastest time for each program.

Program	Gibbon-AoS	Gibbon-SoA	GHC
Compiler	0.3085	**0.2462**	2.007
DBQuery	0.2776	**0.2623**	2.439
DecisionTree	**0.5336**	0.6464	4.107
DomTree	0.1958	**0.1448**	1.189
KDTree	0.1801	**0.1413**	1.695
LinearListReduction	0.0306	**0.0052**	1.261
List	**0.3830**	0.4506	17.09
MonoTree	**0.1184**	0.1310	1.513
ObjectGraph	**0.2550**	0.3919	2.270
OctTree_sumMass	0.0415	**0.0306**	3.220
OctTree_sumEnergy	0.0451	**0.0352**	5.967
OctTree_countActive	0.0376	**0.0302**	5.340
OctTree_countParticles	0.0360	**0.0291**	5.278
OctTree_barnesHutPotential	**0.0432**	0.0445	5.207
OctTree_fmmPotential	0.1084	**0.1021**	6.092
OctTree_scaleEnergy	**0.1502**	0.1761	5.827
OctTree_clearFlags	**0.1543**	0.1724	5.299
PiecewiseFunctions	0.3851	**0.3640**	3.126
TernaryTree	0.1300	**0.1164**	1.117
Trie	0.2232	**0.2127**	1.699
ColorOctree	0.0893	**0.0811**	7.180
Geomean	0.1272	**0.1114**	3.279

Table 7: GHC speedups vs Gibbon mutable variants. Each entry is total median runtime speedup over all passes for one iteration. *GHC/AoS* and *GHC/SoA* are reported.

Program	GHC/AoS	GHC/SoA
Compiler	6.50×	8.15×
DBQuery	8.79×	9.30×
DecisionTree	7.70×	6.35×
DomTree	6.07×	8.21×
KDTree	9.41×	12.00×
LinearListReduction	41.23×	244.26×
List	44.61×	37.92×
MonoTree	12.78×	11.55×
ObjectGraph	8.90×	5.79×
OctTree_sumMass	77.63×	105.36×
OctTree_sumEnergy	132.30×	169.70×
OctTree_countActive	141.91×	177.03×
OctTree_countParticles	146.66×	181.08×
OctTree_barnesHutPotential	120.39×	117.00×
OctTree_fmmPotential	56.20×	59.69×
OctTree_scaleEnergy	38.80×	33.10×
OctTree_clearFlags	34.33×	30.74×
PiecewiseFunctions	8.12×	8.59×
TernaryTree	8.59×	9.60×
Trie	7.61×	7.99×
ColorOctree	80.37×	88.58×
Geomean	25.78×	29.43×

Table 8: Per-pass performance for `OctTree`, OctTree SoA uses 9 buffers; ColorOctree SoA uses 18 buffers (mutable + immutable cursors). Times are median per iteration (s); ± shows standard error of the mean across –iterate runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup >1× means SoA is faster. OOM = out of memory.

Pass	T	Uses	Dead%	Am	Ai	Sm	Si	Am/Sm	Ai/Am	Ai/Sm	Ai/Si
barnesHutPotential	F	11/16	31%	**0.0432**	0.0594	0.0445	–	0.97×	**1.37×**	**1.34×**	–
clearFlags	M	15/16	6%	0.1543	**0.1508**	0.1724	–	0.90×	0.98×	0.88×	–
countActive	F	10/16	38%	0.0376	0.0472	**0.0302**	–	**1.25×**	**1.25×**	**1.56×**	–
countParticles	F	8/16	50%	0.0360	0.0564	**0.0291**	–	**1.23×**	**1.57×**	**1.94×**	–
fmmPotential	F	12/16	25%	0.1084	**0.0986**	0.1021	–	1.06×	0.91×	0.97×	–
scaleEnergy	M	16/16	0%	**0.1502**	0.1535	0.1761	–	0.85×	1.02×	0.87×	–
sumEnergy	F	12/16	25%	0.0451	0.1159	**0.0352**	–	**1.28×**	**2.57×**	**3.30×**	–
sumMass	F	10/16	38%	0.0415	0.0436	**0.0306**	–	**1.36×**	1.05×	**1.43×**	–
paletteEntriesQuantized	F	13/25	48%	0.0436	0.0605	**0.0382**	–	**1.14×**	**1.39×**	**1.59×**	–
quantizationErrorProxy	F	10/25	60%	0.0457	0.0667	**0.0429**	–	1.07×	**1.46×**	**1.56×**	–
Total				0.7057	0.8527	**0.7011**	–	1.01×	**1.21×**	**1.22×**	–
Geomean								1.10×	**1.30×**	**1.43×**	–

Table 9: Per-pass PAPI counters for `OctTree` (A/S). Each cell is median counter value pair per pass. Counter headers are abbreviated for compactness: CYC=CPU cycles, L1D/L1I/L2D/L2I=cache misses, LLC=LLC load misses.

Pass	T	CYC (A/S)	INS (A/S)	L1D (A/S)	L1I (A/S)	L2D (A/S)	L2I (A/S)	LLC (A/S)
barnesHutPotential	F	**2.1e+08**/2.2e+08	**8.2e+08**/1.0e+09	**7.3e+04**/5.7e+06	5.0e+02/**4.9e+02**	**2.1e+03**/1.1e+05	3.5e+02/**3.1e+02**	6.1e+06/**1.7e+06**
clearFlags	M	**3.7e+08**/4.2e+08	**8.0e+08**/1.7e+09	**1.0e+06**/2.2e+06	**1.6e+05**/4.1e+05	**1.2e+04**/3.7e+04	**4.4e+03**/8.7e+03	5.7e+06/**2.4e+06**
countActive	F	1.8e+08/**1.5e+08**	5.8e+08/**5.8e+08**	2.5e+06/**3.6e+05**	4.5e+02/**3.6e+02**	7.1e+04/**1.5e+04**	3.1e+02/**1.9e+02**	6.1e+06/**3.7e+05**
countParticles	F	1.7e+08/**1.4e+08**	5.7e+08/**5.7e+08**	1.0e+06/**3.1e+05**	4.6e+02/**3.2e+02**	**3.9e+03**/7.8e+03	3.4e+02/**1.7e+02**	6.1e+06/**1.1e+05**
fmmPotential	F	5.4e+08/**5.1e+08**	**1.6e+09**/1.8e+09	7.9e+06/**7.9e+06**	1.0e+03/**8.9e+02**	**1.4e+05**/1.5e+05	8.8e+02/**7.5e+02**	4.5e+06/**2.0e+06**
scaleEnergy	M	**3.5e+08**/4.4e+08	**9.1e+08**/1.9e+09	**1.4e+06**/2.3e+06	**2.4e+05**/4.1e+05	**9.0e+03**/3.6e+04	**5.9e+03**/9.5e+03	5.7e+06/**2.5e+06**
sumEnergy	F	2.2e+08/**1.7e+08**	**7.7e+08**/8.9e+08	7.4e+06/**2.8e+06**	6.6e+02/**4.1e+02**	2.7e+05/**6.2e+04**	4.9e+02/**2.4e+02**	5.9e+06/**2.6e+06**
sumMass	F	2.0e+08/**1.5e+08**	**5.7e+08**/6.3e+08	7.3e+06/**2.4e+06**	4.8e+02/**4.2e+02**	2.9e+05/**4.5e+04**	3.5e+02/**2.7e+02**	6.2e+06/**1.2e+06**
paletteEntriesQuantized	F	2.1e+08/**1.8e+08**	**6.7e+08**/7.6e+08	5.3e+06/**2.6e+06**	7.2e+02/**6.2e+02**	**1.2e+04**/7.3e+04	5.4e+02/**4.6e+02**	9.2e+06/**1.2e+06**
quantizationErrorProxy	F	2.3e+08/**2.2e+08**	**9.7e+08**/1.2e+09	4.3e+06/**1.6e+06**	**1.4e+02**/3.7e+02	**1.6e+04**/3.4e+04	**1.2e+02**/3.5e+02	9.3e+06/**2.9e+06**
Total		2.7e+09/2.6e+09	8.2e+09/1.1e+10	3.8e+07/2.8e+07	4.0e+05/8.2e+05	8.3e+05/5.7e+05	1.4e+04/2.1e+04	6.5e+07/1.7e+07

Table 10: Per-pass performance for `Compiler`, ADT has 9 fields, SoA uses 8 buffers (mutable + immutable cursors). Times are median per iteration (s); ± shows standard error of the mean across –iterate runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup >1× means SoA is faster. OOM = out of memory.

Pass	T	Uses	Dead%	Am	Ai	Sm	Si	Am/Sm	Ai/Am	Ai/Sm	Ai/Si
blockCountPass	F	2/9	78%	0.0131	0.0794	**0.0036**	*OOM*	**3.62×**	**6.05×**	**21.89×**	–
branchStatsPass	F	2/9	78%	0.0118	0.0795	**0.0046**	*OOM*	**2.58×**	**6.72×**	**17.33×**	–
castInstCountPass	F	3/9	67%	0.0180	0.0793	**0.0021**	*OOM*	**8.64×**	**4.42×**	**38.18×**	–
has cycle	F	4/9	56%	0.0344	0.0926	**0.0326**	*OOM*	1.05×	**2.69×**	**2.84×**	–
instCountPass	F	2/9	78%	0.0154	0.0791	**0.0041**	*OOM*	**3.71×**	**5.15×**	**19.08×**	–
latencyModelPass	F	3/9	67%	0.0120	0.0799	**0.0035**	*OOM*	**3.48×**	**6.64×**	**23.15×**	–
memoryOpStatsPass	F	3/9	67%	0.0118	0.0795	**0.0056**	*OOM*	**2.10×**	**6.75×**	**14.19×**	–
stripSideEffectsPass	M	7/9	22%	**0.0800**	0.1856	0.0906	*OOM*	0.88×	**2.32×**	**2.05×**	–
targetReturnPass	M	9/9	0%	**0.0797**	0.2040	0.0899	*OOM*	0.89×	**2.56×**	**2.27×**	–
throughputModelPass	F	3/9	67%	0.0120	0.0800	**0.0030**	*OOM*	**4.06×**	**6.66×**	**27.01×**	–
verifyIR	F	9/9	0%	0.0203	0.0704	**0.0066**	*OOM*	**3.10×**	**3.46×**	**10.75×**	–
Total				0.3085	1.109	**0.2462**	–	**1.25×**	**3.60×**	**4.51×**	–
Geomean								**2.49×**	**4.50×**	**11.20×**	–

Table 11: Per-pass PAPI counters for `Compiler` (A/S). Each cell is median counter value pair per pass. Counter headers are abbreviated for compactness: CYC=CPU cycles, L1D/L1I/L2D/L2I=cache misses, LLC=LLC load misses.

Pass	T	CYC (A/S)	INS (A/S)	L1D (A/S)	L1I (A/S)	L2D (A/S)	L2I (A/S)	LLC (A/S)
blockCountPass	F	4.5e+07/**9.8e+06**	5.1e+07/**4.5e+07**	1.3e+06/**7.3e+03**	2.5e+02/**9.4e+01**	2.7e+05/**1.1e+03**	2.2e+02/**8.0e+01**	4.7e+06/**1.8e+04**
branchStatsPass	F	6.0e+07/**1.6e+07**	**7.9e+07**/8.6e+07	8.2e+05/**2.2e+04**	3.4e+02/**9.7e+01**	4.9e+04/**3.4e+03**	3.1e+02/**8.6e+01**	4.7e+06/**4.8e+05**
castInstCountPass	F	9.1e+07/**6.6e+06**	5.1e+07/**3.9e+07**	**1.2e+04**/3.0e+04	1.3e+02/**9.4e+01**	**8.0e+02**/1.2e+03	1.1e+02/**6.7e+01**	4.7e+06/**1.4e+04**
has cycle	F	1.3e+08/**8.9e+07**	**1.1e+08**/1.5e+08	2.3e+06/**4.9e+05**	**3.2e+04**/8.7e+04	1.4e+05/**2.2e+04**	1.9e+03/**5.1e+02**	5.0e+06/**1.5e+06**
instCountPass	F	3.8e+07/**1.1e+07**	**5.6e+07**/5.7e+07	1.4e+06/**1.6e+04**	3.4e+02/**1.4e+02**	3.2e+05/**1.2e+03**	3.0e+02/**1.2e+02**	4.7e+06/**4.4e+04**
latencyModelPass	F	6.1e+07/**1.3e+07**	**6.2e+07**/6.9e+07	1.1e+06/**3.4e+04**	2.9e+02/**9.6e+01**	1.9e+05/**3.8e+03**	2.6e+02/**8.2e+01**	4.8e+06/**6.2e+05**
memoryOpStatsPass	F	5.6e+07/**1.7e+07**	**8.5e+07**/9.1e+07	6.5e+05/**2.7e+04**	2.6e+02/**1.2e+02**	3.1e+04/**3.3e+03**	2.3e+02/**1.1e+02**	4.7e+06/**4.8e+05**
stripSideEffectsPass	M	**1.5e+08**/2.0e+08	**1.8e+08**/6.0e+08	1.9e+06/**1.1e+06**	**1.4e+05**/1.5e+05	**7.7e+03**/1.6e+04	**5.5e+03**/8.5e+03	4.3e+06/**1.2e+06**
targetReturnPass	M	**1.5e+08**/2.0e+08	**1.7e+08**/6.3e+08	1.6e+06/**1.1e+06**	**9.8e+04**/2.0e+05	**5.6e+03**/2.1e+04	**3.0e+03**/2.0e+04	3.7e+06/**1.4e+06**
throughputModelPass	F	6.1e+07/**1.5e+07**	**6.2e+07**/6.9e+07	1.1e+06/**3.4e+04**	2.7e+02/**1.2e+02**	1.9e+05/**3.6e+03**	2.7e+02/**8.5e+01**	4.8e+06/**6.8e+05**
verifyIR	F	3.6e+07/**1.5e+07**	**7.4e+07**/8.6e+07	1.1e+06/**3.9e+05**	9.2e+02/**7.5e+02**	3.4e+05/**1.6e+04**	8.0e+02/**5.9e+02**	4.6e+06/**4.5e+05**
Total		8.8e+08/5.9e+08	9.9e+08/1.9e+09	1.3e+07/3.3e+06	2.7e+05/4.4e+05	1.5e+06/9.2e+04	1.3e+04/3.1e+04	5.1e+07/6.9e+06

Table 12: Per-pass PAPI counters for `Compiler` (Ai/Si). Each cell is median counter value pair per pass. Counter headers are abbreviated for compactness: CYC=CPU cycles, L1D/L1I/L2D/L2I=cache misses, LLC=LLC load misses.

Pass	T	CYC (Ai/Si)	INS (Ai/Si)	L1D (Ai/Si)	L1I (Ai/Si)	L2D (Ai/Si)	L2I (Ai/Si)	LLC (Ai/Si)
blockCountPass	F	4.0e+08/–	1.6e+08/–	2.2e+06/–	3.7e+02/–	9.4e+04/–	3.6e+02/–	1.4e+07/–
branchStatsPass	F	4.0e+08/–	1.8e+08/–	2.1e+06/–	3.9e+02/–	4.3e+04/–	3.8e+02/–	1.4e+07/–
castInstCountPass	F	4.0e+08/–	1.8e+08/–	2.1e+06/–	4.0e+02/–	4.6e+04/–	3.7e+02/–	1.4e+07/–
has cycle	F	4.7e+08/–	2.1e+08/–	2.5e+06/–	3.8e+02/–	7.0e+04/–	3.6e+02/–	1.7e+07/–
instCountPass	F	4.0e+08/–	1.6e+08/–	2.2e+06/–	5.2e+02/–	9.9e+04/–	5.1e+02/–	1.4e+07/–
latencyModelPass	F	4.0e+08/–	1.7e+08/–	2.1e+06/–	4.8e+02/–	4.6e+04/–	4.7e+02/–	1.4e+07/–
memoryOpStatsPass	F	4.0e+08/–	1.9e+08/–	2.0e+06/–	6.3e+02/–	4.2e+04/–	6.0e+02/–	1.4e+07/–
stripSideEffectsPass	M	6.7e+08/–	3.3e+08/–	2.0e+06/–	1.6e+05/–	2.2e+05/–	1.1e+04/–	2.4e+07/–
targetReturnPass	M	6.6e+08/–	3.2e+08/–	2.3e+06/–	2.3e+05/–	2.3e+05/–	1.7e+04/–	2.2e+07/–
throughputModelPass	F	4.1e+08/–	1.7e+08/–	2.1e+06/–	3.3e+02/–	4.5e+04/–	3.2e+02/–	1.4e+07/–
verifyIR	F	3.5e+08/–	1.2e+08/–	2.4e+06/–	8.1e+02/–	2.0e+05/–	7.3e+02/–	1.1e+07/–
Total		5.0e+09/–	2.2e+09/–	2.4e+07/–	3.9e+05/–	1.1e+06/–	3.2e+04/–	1.7e+08/–

Table 13: Per-pass performance for `DBQuery`, ADT has 15 fields, SoA uses 13 buffers (mutable + immutable cursors). Times are median per iteration (s); ± shows standard error of the mean across –iterate runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup >1× means SoA is faster. OOM = out of memory.

Pass	T	Uses	Dead%	Am	Ai	Sm	Si	Am/Sm	Ai/Am	Ai/Sm	Ai/Si
clearQueryFlags	M	14/15	7%	**0.0685**	0.0742	0.0838	0.2949	0.82×	1.08×	0.88×	0.25×
countJoins	F	3/15	80%	0.0196	0.0286	**0.0112**	0.0854	**1.75**×	**1.46**×	**2.56**×	0.34×
filterSelectivitySkew	F	5/15	67%	0.0197	0.0287	**0.0123**	0.0860	**1.60**×	**1.46**×	**2.34**×	0.33×
hashJoinPressure	F	5/15	67%	0.0205	0.0328	**0.0125**	0.0859	**1.64**×	**1.60**×	**2.62**×	0.38×
scaleCosts	M	15/15	0%	**0.0689**	0.0759	0.0843	0.2978	0.82×	**1.10**×	0.90×	0.25×
sumCost	F	6/15	60%	0.0319	0.0413	**0.0210**	0.0894	**1.52**×	**1.30**×	**1.97**×	0.46×
sumMemory	F	6/15	60%	0.0198	0.0288	**0.0144**	0.0876	**1.38**×	**1.45**×	**2.00**×	0.33×
sumRows	F	6/15	60%	0.0287	0.0309	**0.0227**	0.0870	**1.26**×	1.08×	**1.36**×	0.36×
Total				0.2776	0.3412	**0.2623**	1.114	1.06×	**1.23**×	**1.30**×	0.31×
Geomean								**1.30**×	**1.30**×	**1.69**×	0.33×

Table 14: Per-pass PAPI counters for `DBQuery` (A/S). Each cell is median counter value pair per pass. Counter headers are abbreviated for compactness: CYC=CPU cycles, L1D/L1I/L2D/L2I=cache misses, LLC=LLC load misses.

Pass	T	CYC (A/S)	INS (A/S)	L1D (A/S)	L1I (A/S)	L2D (A/S)	L2I (A/S)	LLC (A/S)
clearQueryFlags	M	**1.6e+08**/2.2e+08	**1.9e+08**/7.2e+08	**2.4e+05**/1.5e+06	**8.1e+04**/2.5e+05	2.9e+04/**2.8e+04**	**3.0e+03**/1.6e+04	2.3e+06/**9.3e+05**
countJoins	F	1.0e+08/**5.7e+07**	7.4e+07/**7.1e+07**	7.1e+05/**2.6e+04**	4.8e+02/**2.1e+02**	1.7e+05/**2.1e+03**	4.1e+02/**1.6e+02**	2.8e+06/**5.8e+04**
filterSelectivitySkew	F	9.9e+07/**6.2e+07**	**1.0e+08**/1.1e+08	5.2e+05/**6.3e+04**	3.4e+02/**1.4e+02**	1.2e+05/**5.9e+03**	3.3e+02/**1.2e+02**	2.7e+06/**3.3e+05**
hashJoinPressure	F	1.0e+08/**6.4e+07**	**8.4e+07**/9.9e+07	7.0e+05/**9.3e+04**	3.6e+02/**1.5e+02**	1.8e+05/**7.3e+03**	3.3e+02/**1.3e+02**	2.7e+06/**1.4e+05**
scaleCosts	M	**1.6e+08**/2.2e+08	**2.2e+08**/7.5e+08	**2.3e+05**/1.1e+06	**7.2e+04**/2.2e+05	3.3e+04/**2.6e+04**	**3.5e+03**/1.5e+04	2.5e+06/**1.0e+06**
sumCost	F	8.4e+07/**7.1e+07**	**9.0e+07**/1.2e+08	7.2e+05/**6.9e+05**	7.4e+02/**3.6e+02**	1.6e+05/**1.9e+04**	6.1e+02/**2.6e+02**	2.4e+06/**3.3e+05**
sumMemory	F	1.0e+08/**7.3e+07**	**9.0e+07**/1.2e+08	5.7e+05/**8.7e+04**	4.6e+02/**1.8e+02**	1.3e+05/**1.2e+04**	4.5e+02/**1.6e+02**	2.7e+06/**3.9e+05**
sumRows	F	1.3e+08/**1.1e+08**	**2.1e+08**/2.3e+08	2.5e+05/**1.1e+05**	1.9e+02/**1.6e+02**	2.2e+04/**7.5e+03**	1.7e+02/**1.5e+02**	2.4e+06/**3.2e+05**
Total		9.5e+08/8.8e+08	1.1e+09/2.2e+09	4.0e+06/3.6e+06	1.6e+05/4.7e+05	8.4e+05/1.1e+05	8.8e+03/3.2e+04	2.1e+07/3.5e+06

Table 15: Per-pass PAPI counters for `DBQuery` (Ai/Si). Each cell is median counter value pair per pass. Counter headers are abbreviated for compactness: CYC=CPU cycles, L1D/L1I/L2D/L2I=cache misses, LLC=LLC load misses.

Pass	T	CYC (Ai/Si)	INS (Ai/Si)	L1D (Ai/Si)	L1I (Ai/Si)	L2D (Ai/Si)	L2I (Ai/Si)	LLC (Ai/Si)
clearQueryFlags	M	**2.0e+08**/1.3e+09	**3.2e+08**/2.4e+09	**5.9e+05**/1.9e+07	**6.3e+04**/4.2e+05	**1.5e+04**/4.7e+04	**1.5e+03**/3.6e+04	2.1e+06/**9.3e+05**
countJoins	F	**1.5e+08**/4.3e+08	**1.6e+08**/7.7e+08	1.9e+06/**3.4e+05**	**2.8e+02**/9.3e+02	1.1e+05/**1.3e+04**	2.2e+02/**1.9e+02**	2.4e+06/**8.3e+04**
filterSelectivitySkew	F	**1.5e+08**/4.4e+08	**2.0e+08**/7.9e+08	1.5e+06/**8.9e+05**	**3.0e+02**/9.3e+02	7.7e+04/**1.7e+04**	2.8e+02/**2.5e+02**	2.3e+06/**2.2e+05**
hashJoinPressure	F	**1.7e+08**/4.4e+08	**1.8e+08**/7.7e+08	1.6e+06/**6.7e+05**	**2.6e+02**/6.3e+02	9.3e+04/**1.6e+04**	2.3e+02/**1.6e+02**	2.3e+06/**1.6e+05**
scaleCosts	M	**2.0e+08**/1.3e+09	**3.5e+08**/2.4e+09	**5.2e+05**/2.3e+07	**1.0e+05**/4.7e+05	**1.9e+04**/5.6e+04	**2.7e+03**/5.1e+04	2.3e+06/**1.2e+06**
sumCost	F	**1.4e+08**/4.5e+08	**1.8e+08**/7.8e+08	2.2e+06/**1.4e+06**	**4.8e+02**/1.4e+03	1.2e+05/**2.3e+04**	**4.0e+02**/7.4e+02	2.2e+06/**3.3e+05**
sumMemory	F	**1.5e+08**/4.4e+08	**1.8e+08**/7.8e+08	1.6e+06/**1.2e+06**	**3.6e+02**/8.4e+02	7.7e+04/**2.0e+04**	**3.2e+02**/6.1e+02	2.3e+06/**3.3e+05**
sumRows	F	**1.5e+08**/4.4e+08	**2.3e+08**/8.3e+08	1.5e+06/**1.2e+06**	**6.4e+02**/1.1e+03	6.7e+04/**2.2e+04**	**6.1e+02**/6.7e+02	2.3e+06/**3.5e+05**
Total		1.3e+09/5.2e+09	1.8e+09/9.5e+09	1.1e+07/4.8e+07	1.7e+05/9.0e+05	5.7e+05/2.2e+05	6.3e+03/9.0e+04	1.8e+07/3.6e+06

Table 16: Per-pass performance for DecisionTree, ADT has 7 fields, SoA uses 6 buffers (mutable + immutable cursors). Times are median per iteration (s); ± shows standard error of the mean across –iterate runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup >1× means SoA is faster. OOM = out of memory.

Pass	T	Uses	Dead%	Am	Ai	Sm	Si	Am/Sm	Ai/Am	Ai/Sm	Ai/Si
classify Batch	F	5/7	29%	**1.90e-05**	5.50e-05	6.30e-05	7.00e-05	0.30×	**2.89**×	0.87×	0.79×
classify Depth	F	4/7	43%	–	–	–	–	–	–	–	–
classify tree	F	5/7	29%	–	**2.00e-06**	**2.00e-06**	**2.00e-06**	–	–	1.00×	1.00×
countFeatureUses	F	3/7	57%	**0.0604**	0.1034	0.0746	0.3079	0.81×	**1.71**×	**1.39**×	0.34×
countLeaves	F	2/7	71%	**0.0578**	0.0991	0.0675	0.3076	0.86×	**1.71**×	**1.47**×	0.32×
countNodes	F	2/7	71%	**0.0595**	0.1022	0.0685	0.3096	0.87×	**1.72**×	**1.49**×	0.33×
countSmallLeaves	F	3/7	57%	**0.0606**	0.0997	0.0729	0.2850	0.83×	**1.65**×	**1.37**×	0.35×
inferenceCost	F	2/7	71%	**0.0581**	0.2161	0.0681	0.3139	0.85×	**3.72**×	**3.17**×	0.69×
sumImpurity	F	3/7	57%	**0.0595**	0.1037	0.0737	0.3152	0.81×	**1.74**×	**1.41**×	0.33×
sumPathLengths	F	3/7	57%	**0.0599**	0.1035	0.0778	0.3031	0.77×	**1.73**×	**1.33**×	0.34×
sumSamples	F	3/7	57%	**0.0591**	0.0997	0.0750	0.2982	0.79×	**1.69**×	**1.33**×	0.33×
treeDepth	F	2/7	71%	**0.0588**	0.1014	0.0681	0.3138	0.86×	**1.73**×	**1.49**×	0.32×
Total				**0.5336**	1.029	0.6464	2.754	0.83×	**1.93**×	**1.59**×	0.37×
Geomean								0.75×	**1.95**×	**1.41**×	0.43×

Table 17: Per-pass PAPI counters for DecisionTree (A/S). Each cell is median counter value pair per pass. Counter headers are abbreviated for compactness: CYC=CPU cycles, L1D/L1I/L2D/L2I=cache misses, LLC=LLC load misses.

Pass	T	CYC (A/S)	INS (A/S)	L1D (A/S)	L1I (A/S)	L2D (A/S)	L2I (A/S)	LLC (A/S)
classify Batch	F	**5.0e+04**/2.6e+05	**1.7e+05**/2.7e+05	**4.2e+01**/1.6e+03	**2.4e+02**/3.9e+02	**6.0e+00**/1.2e+03	**2.3e+01**/7.1e+01	**6.2e+01**/6.0e+03
classify Depth	F	**8.7e+02**/2.7e+03	**1.3e+03**/2.3e+03	**8.0e+00**/2.4e+01	**3.9e+01**/4.6e+01	1.0e+00/**0.0e+00**	**8.0e+00**/2.6e+01	**7.0e+00**/4.3e+01
classify tree	F	**1.6e+03**/1.3e+04	**1.2e+03**/2.1e+03	**1.8e+01**/1.2e+02	**2.7e+01**/4.8e+01	**7.0e+00**/9.0e+01	**3.0e+00**/1.2e+01	**4.9e+01**/4.0e+02
countFeatureUses	F	**3.1e+08**/3.8e+08	**7.4e+08**/1.1e+09	**9.0e+06**/1.1e+07	**1.2e+03**/2.2e+03	**4.7e+04**/1.5e+05	**1.2e+03**/2.2e+03	**1.8e+07**/2.0e+07
countLeaves	F	**2.9e+08**/3.4e+08	**6.5e+08**/9.3e+08	**8.5e+06**/1.0e+07	**1.1e+03**/2.6e+03	**8.7e+04**/1.6e+05	**1.1e+03**/2.5e+03	**1.8e+07**/1.9e+07
countNodes	F	**2.9e+08**/3.4e+08	**6.5e+08**/9.3e+08	**8.5e+06**/1.0e+07	1.7e+03/**1.6e+03**	**1.1e+05**/1.5e+05	1.5e+03/**1.4e+03**	**1.8e+07**/1.9e+07
countSmallLeaves	F	**3.1e+08**/3.7e+08	**7.1e+08**/1.0e+09	**8.8e+06**/1.2e+07	**9.7e+02**/2.4e+03	**3.6e+04**/1.5e+05	**9.6e+02**/2.3e+03	**1.8e+07**/2.0e+07
inferenceCost	F	**2.9e+08**/3.5e+08	**6.9e+08**/9.8e+08	**8.6e+06**/1.0e+07	**9.3e+02**/2.6e+03	**8.4e+04**/1.6e+05	**9.0e+02**/2.5e+03	**1.8e+07**/1.9e+07
sumImpurity	F	**3.0e+08**/3.7e+08	**6.8e+08**/1.0e+09	**8.6e+06**/1.1e+07	**8.4e+02**/2.3e+03	**5.8e+04**/1.4e+05	**8.2e+02**/2.2e+03	**1.8e+07**/2.0e+07
sumPathLengths	F	**3.0e+08**/3.9e+08	**7.8e+08**/1.1e+09	**9.3e+06**/1.3e+07	**1.6e+03**/2.7e+03	**4.4e+04**/1.5e+05	**1.6e+03**/2.7e+03	**1.8e+07**/1.9e+07
sumSamples	F	**3.0e+08**/3.8e+08	**6.6e+08**/9.7e+08	**8.4e+06**/1.2e+07	**1.1e+03**/2.5e+03	**4.9e+04**/1.5e+05	**1.1e+03**/2.4e+03	**1.8e+07**/2.0e+07
treeDepth	F	**3.0e+08**/3.5e+08	**6.9e+08**/9.8e+08	**8.9e+06**/1.0e+07	**9.2e+02**/2.2e+03	**7.0e+04**/1.6e+05	**8.7e+02**/2.2e+03	**1.8e+07**/1.9e+07
Total		2.7e+09/3.3e+09	6.3e+09/9.0e+09	7.9e+07/1.0e+08	1.1e+04/2.2e+04	5.9e+05/1.4e+06	1.0e+04/2.1e+04	1.6e+08/1.7e+08

Table 18: Per-pass PAPI counters for DecisionTree (Ai/Si). Each cell is median counter value pair per pass. Counter headers are abbreviated for compactness: CYC=CPU cycles, L1D/L1I/L2D/L2I=cache misses, LLC=LLC load misses.

Pass	T	CYC (Ai/Si)	INS (Ai/Si)	L1D (Ai/Si)	L1I (Ai/Si)	L2D (Ai/Si)	L2I (Ai/Si)	LLC (Ai/Si)
classify Batch	F	**1.9e+05**/2.9e+05	**2.0e+05**/2.9e+05	**6.4e+02**/1.7e+03	3.8e+02/**3.6e+02**	**4.3e+02**/1.1e+03	6.7e+01/**5.6e+01**	**2.0e+03**/5.7e+03
classify Depth	F	**1.7e+03**/2.5e+03	**1.5e+03**/2.3e+03	**2.3e+01**/3.5e+01	5.7e+01/**4.9e+01**	0.0e+00/0.0e+00	3.3e+01/**2.8e+01**	**1.7e+01**/8.3e+01
classify tree	F	**6.9e+03**/1.3e+04	**1.5e+03**/2.3e+03	**4.9e+01**/1.4e+02	**3.2e+01**/4.1e+01	**3.6e+01**/9.5e+01	**1.0e+01**/1.5e+01	**1.2e+02**/4.6e+02
countFeatureUses	F	**5.2e+08**/1.6e+09	**1.4e+09**/2.9e+09	**6.0e+05**/1.7e+06	**5.0e+02**/2.4e+03	**8.8e+03**/1.8e+05	**4.7e+02**/2.4e+03	1.4e+07/**1.1e+07**
countLeaves	F	**5.0e+08**/1.6e+09	**1.3e+09**/2.8e+09	**5.2e+05**/1.5e+06	**3.1e+02**/2.8e+03	**9.8e+03**/1.8e+05	**2.6e+02**/2.8e+03	1.5e+07/**1.0e+07**
countNodes	F	**5.1e+08**/1.6e+09	**1.3e+09**/2.9e+09	2.9e+06/**1.8e+06**	**9.0e+02**/2.5e+03	**8.1e+04**/1.9e+05	**8.1e+02**/2.4e+03	1.4e+07/**1.0e+07**
countSmallLeaves	F	**5.1e+08**/1.4e+09	**1.3e+09**/2.8e+09	**3.4e+05**/2.2e+06	**4.8e+02**/1.3e+03	**8.2e+03**/1.8e+05	**4.7e+02**/1.3e+03	1.5e+07/**1.2e+07**
inferenceCost	F	**5.1e+08**/1.6e+09	**1.4e+09**/2.9e+09	5.3e+06/**1.7e+06**	**6.9e+02**/2.1e+03	**1.5e+05**/1.9e+05	**6.8e+02**/2.1e+03	1.2e+07/**1.0e+07**
sumImpurity	F	**5.3e+08**/1.6e+09	**1.4e+09**/2.9e+09	**6.3e+05**/1.9e+06	**6.2e+02**/1.8e+03	**9.2e+03**/1.6e+05	**6.1e+02**/1.8e+03	1.4e+07/**1.1e+07**
sumPathLengths	F	**5.3e+08**/1.5e+09	**1.4e+09**/3.0e+09	**3.4e+05**/2.2e+06	**3.5e+02**/7.1e+02	**6.3e+03**/1.7e+05	**3.1e+02**/6.9e+02	1.4e+07/**1.1e+07**
sumSamples	F	**5.0e+08**/1.5e+09	**1.3e+09**/2.9e+09	**3.7e+05**/2.0e+06	**4.0e+02**/3.3e+03	**7.8e+03**/1.8e+05	**3.8e+02**/3.2e+03	1.5e+07/**1.1e+07**
treeDepth	F	**5.1e+08**/1.6e+09	**1.4e+09**/2.9e+09	**5.2e+05**/1.7e+06	**5.9e+02**/2.7e+03	**9.3e+03**/1.9e+05	**5.7e+02**/2.7e+03	1.5e+07/**1.0e+07**
Total		4.6e+09/1.4e+10	1.2e+10/2.6e+10	1.2e+07/1.7e+07	5.3e+03/2.0e+04	2.9e+05/1.6e+06	4.7e+03/1.9e+04	1.3e+08/9.8e+07

Table 19: Per-pass performance for DomTree, ADT has 15 fields, SoA uses 14 buffers (mutable + immutable cursors). Times are median per iteration (s); ± shows standard error of the mean across –iterate runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup >1× means SoA is faster. OOM = out of memory.

Pass	T	Uses	Dead%	Am	Ai	Sm	Si	Am/Sm	Ai/Am	Ai/Sm	Ai/Si
SumArea	F	6/15	60%	0.0358	0.0742	**0.0203**	0.0901	**1.76**×	**2.07**×	**3.65**×	0.82×
computeWidths	M	14/15	7%	**0.0298**	0.0302	0.0449	0.0884	0.66×	1.02×	0.67×	0.34×
count styled	F	3/15	80%	0.0337	0.0696	**0.0117**	0.0866	**2.87**×	**2.07**×	**5.95**×	0.80×
find max Bottom	F	5/15	67%	0.0339	0.0694	**0.0188**	0.0867	**1.80**×	**2.05**×	**3.68**×	0.80×
scaleLayout	M	15/15	0%	**0.0290**	0.0305	0.0372	0.0893	0.78×	1.05×	0.82×	0.34×
sumTextWidth	F	3/15	80%	0.0337	0.0702	**0.0119**	0.0859	**2.84**×	**2.08**×	**5.91**×	0.82×
Total				0.1958	0.3441	**0.1448**	0.5270	**1.35**×	**1.76**×	**2.38**×	0.65×
Geomean								**1.54**×	**1.64**×	**2.53**×	0.61×

Table 20: Per-pass PAPI counters for DomTree (A/S). Each cell is median counter value pair per pass. Counter headers are abbreviated for compactness: CYC=CPU cycles, L1D/L1I/L2D/L2I=cache misses, LLC=LLC load misses.

Pass	T	CYC (A/S)	INS (A/S)	L1D (A/S)	L1I (A/S)	L2D (A/S)	L2I (A/S)	LLC (A/S)
SumArea	F	1.8e+08/**9.7e+07**	**2.9e+08**/4.9e+08	2.5e+06/**7.9e+05**	1.1e+03/**3.9e+02**	2.0e+05/**1.8e+04**	9.1e+02/**2.8e+02**	1.4e+07/**1.8e+06**
computeWidths	M	**6.5e+07**/1.4e+08	**1.4e+08**/4.6e+08	**1.2e+06**/1.5e+06	5.3e+04/**5.1e+04**	**5.0e+03**/9.5e+03	**1.6e+03**/1.6e+04	1.3e+06/**4.5e+05**
count styled	F	1.7e+08/**5.9e+07**	**2.6e+08**/2.6e+08	2.1e+06/**3.3e+04**	8.8e+02/**1.8e+02**	3.1e+05/**5.0e+03**	8.4e+02/**1.8e+02**	1.4e+07/**7.1e+05**
find max Bottom	F	1.7e+08/**9.5e+07**	**2.5e+08**/4.1e+08	2.4e+06/**5.2e+05**	8.9e+02/**1.8e+02**	1.5e+05/**9.2e+03**	8.3e+02/**1.6e+02**	1.4e+07/**1.5e+06**
scaleLayout	M	**6.0e+07**/1.0e+08	**8.8e+07**/2.9e+08	**1.1e+06**/2.3e+06	**5.0e+04**/6.6e+04	2.5e+04/**7.9e+03**	**2.2e+03**/1.5e+04	1.4e+06/**4.5e+05**
sumTextWidth	F	1.7e+08/**5.9e+07**	**2.0e+08**/2.4e+08	2.4e+06/**2.0e+04**	9.6e+02/**1.8e+02**	4.4e+05/**4.3e+03**	9.2e+02/**1.7e+02**	1.4e+07/**7.1e+05**
Total		8.2e+08/5.5e+08	1.2e+09/2.2e+09	1.2e+07/5.1e+06	1.1e+05/1.2e+05	1.1e+06/5.4e+04	7.4e+03/3.2e+04	5.7e+07/5.6e+06

Table 21: Per-pass PAPI counters for DomTree (Ai/Si). Each cell is median counter value pair per pass. Counter headers are abbreviated for compactness: CYC=CPU cycles, L1D/L1I/L2D/L2I=cache misses, LLC=LLC load misses.

Pass	T	CYC (Ai/Si)	INS (Ai/Si)	L1D (Ai/Si)	L1I (Ai/Si)	L2D (Ai/Si)	L2I (Ai/Si)	LLC (Ai/Si)
SumArea	F	**3.7e+08**/4.5e+08	**4.4e+08**/1.8e+09	9.5e+06/**2.1e+06**	**9.3e+02**/1.3e+03	1.1e+05/**3.2e+04**	**8.0e+02**/1.2e+03	9.9e+06/**1.4e+06**
computeWidths	M	**6.8e+07**/3.6e+08	**1.5e+08**/8.9e+08	**1.3e+06**/3.2e+06	**5.3e+04**/3.0e+05	**4.9e+03**/1.4e+04	**1.3e+03**/2.4e+04	1.2e+06/**4.7e+05**
count styled	F	**3.5e+08**/4.4e+08	**4.3e+08**/1.7e+09	1.0e+07/**1.2e+06**	**1.7e+02**/7.4e+02	3.6e+04/**3.6e+04**	**1.6e+02**/5.8e+02	1.0e+07/**4.6e+05**
find max Bottom	F	**3.5e+08**/4.4e+08	**4.6e+08**/1.8e+09	1.0e+07/**1.4e+06**	**1.8e+02**/7.3e+02	**1.5e+04**/2.6e+04	**1.5e+02**/7.1e+02	1.0e+07/**1.0e+06**
scaleLayout	M	**6.8e+07**/3.6e+08	**1.1e+08**/7.6e+08	**9.6e+05**/4.1e+06	**3.7e+04**/1.4e+05	**9.2e+03**/1.4e+04	**1.3e+03**/2.1e+04	1.2e+06/**4.6e+05**
sumTextWidth	F	**3.6e+08**/4.4e+08	**3.9e+08**/1.7e+09	1.1e+07/**1.3e+06**	**2.5e+02**/8.7e+02	8.8e+04/**3.9e+04**	**2.3e+02**/6.8e+02	1.0e+07/**4.5e+05**
Total		1.6e+09/2.5e+09	2.0e+09/8.6e+09	4.2e+07/1.3e+07	9.1e+04/4.5e+05	2.7e+05/1.6e+05	3.9e+03/4.8e+04	4.3e+07/4.3e+06

Table 22: Per-pass performance for `KDTree`, ADT has 17 fields, SoA uses 16 buffers (mutable + immutable cursors). Times are median per iteration (s); ± shows standard error of the mean across –iterate runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup >1× means SoA is faster. OOM = out of memory.

Pass	**T**	**Uses**	**Dead%**	**Am**	**Ai**	**Sm**	**Si**	**Am/Sm**	**Ai/Am**	**Ai/Sm**	**Ai/Si**
Find nearest Neighbour	F	13/17	24%	0.0310	0.0506	**0.0238**	0.1175	**1.30**×	**1.63**×	**2.13**×	0.43×
countInRange tight_box	F	13/17	24%	0.0275	0.0465	**0.0197**	0.1211	**1.40**×	**1.69**×	**2.36**×	0.38×
photonMappingBenchmark	F	12/17	29%	0.0417	0.0473	**0.0407**	0.1290	1.03×	**1.13**×	**1.16**×	0.37×
pointCloudNeighborhood	F	11/17	35%	0.0244	0.0382	**0.0165**	0.1126	**1.48**×	**1.56**×	**2.31**×	0.34×
sumMassInRange	F	14/17	18%	0.0269	0.0402	**0.0199**	0.1204	**1.35**×	**1.50**×	**2.02**×	0.33×
twoPointCorrelation bin_8_16	F	11/17	35%	0.0286	0.0387	**0.0207**	0.1108	**1.38**×	**1.35**×	**1.87**×	0.35×
Total				0.1801	0.2616	**0.1413**	0.7114	**1.27**×	**1.45**×	**1.85**×	0.37×
Geomean								**1.31**×	**1.47**×	**1.93**×	0.37×

Table 23: Per-pass PAPI counters for `KDTree` (A/S). Each cell is median counter value pair per pass. Counter headers are abbreviated for compactness: CYC=CPU cycles, L1D/L1I/L2D/L2I=cache misses, LLC=LLC load misses.

Pass	**T**	**CYC (A/S)**	**INS (A/S)**	**L1D (A/S)**	**L1I (A/S)**	**L2D (A/S)**	**L2I (A/S)**	**LLC (A/S)**
Find nearest Neighbour	F	1.2e+08/**1.1e+08**	**4.7e+08**/6.3e+08	3.6e+06/**2.2e+06**	4.6e+02/**4.4e+02**	**1.6e+04**/3.2e+04	**2.7e+02**/3.2e+02	7.5e+06/**1.5e+06**
countInRange tight_box	F	1.4e+08/**1.0e+08**	**4.0e+08**/5.5e+08	2.1e+06/**1.5e+06**	**2.0e+02**/2.5e+02	**3.2e+03**/1.7e+04	**1.8e+02**/2.3e+02	7.8e+06/**1.6e+06**
photonMappingBenchmark	F	2.1e+08/**2.1e+08**	**1.0e+09**/1.2e+09	**2.2e+06**/2.5e+06	**3.4e+02**/5.6e+02	**3.5e+03**/1.5e+04	**3.2e+02**/5.5e+02	5.7e+06/**1.5e+06**
pointCloudNeighborhood	F	1.2e+08/**8.4e+07**	**3.3e+08**/4.8e+08	2.6e+06/**7.7e+05**	2.4e+02/**1.8e+02**	**4.5e+03**/1.6e+04	2.3e+02/**1.8e+02**	7.8e+06/**1.4e+06**
sumMassInRange	F	1.4e+08/**1.0e+08**	**4.0e+08**/5.7e+08	2.3e+06/**2.3e+06**	**1.6e+02**/2.8e+02	**4.0e+03**/1.9e+04	**1.2e+02**/2.7e+02	7.8e+06/**1.9e+06**
twoPointCorrelation bin_8_16	F	1.5e+08/**1.1e+08**	**4.8e+08**/6.1e+08	2.1e+06/**6.1e+05**	1.6e+02/**1.5e+02**	**5.6e+03**/1.3e+04	1.4e+02/**1.3e+02**	7.7e+06/**1.3e+06**
Total		8.7e+08/7.0e+08	3.1e+09/4.0e+09	1.5e+07/9.9e+06	1.6e+03/1.9e+03	3.6e+04/1.1e+05	1.3e+03/1.7e+03	4.4e+07/9.1e+06

Table 24: Per-pass PAPI counters for `KDTree` (Ai/Si). Each cell is median counter value pair per pass. Counter headers are abbreviated for compactness: CYC=CPU cycles, L1D/L1I/L2D/L2I=cache misses, LLC=LLC load misses.

Pass	**T**	**CYC (Ai/Si)**	**INS (Ai/Si)**	**L1D (Ai/Si)**	**L1I (Ai/Si)**	**L2D (Ai/Si)**	**L2I (Ai/Si)**	**LLC (Ai/Si)**
Find nearest Neighbour	F	**2.3e+08**/5.9e+08	**4.9e+08**/1.3e+09	7.6e+06/**5.2e+06**	**6.8e+02**/2.3e+03	2.8e+05/**2.1e+04**	**4.9e+02**/2.2e+03	5.0e+06/**1.5e+06**
countInRange tight_box	F	**2.4e+08**/6.1e+08	**4.5e+08**/1.3e+09	7.5e+06/**5.8e+06**	**3.5e+02**/1.6e+03	2.8e+05/**2.2e+04**	**3.4e+02**/1.6e+03	5.2e+06/**1.6e+06**
photonMappingBenchmark	F	**2.4e+08**/6.5e+08	**1.1e+09**/2.0e+09	**2.9e+06**/5.9e+06	**3.3e+02**/2.7e+03	**3.3e+03**/2.5e+04	**3.3e+02**/2.7e+03	5.1e+06/**1.5e+06**
pointCloudNeighborhood	F	**1.9e+08**/5.7e+08	**4.1e+08**/1.3e+09	5.1e+06/**4.8e+06**	**2.7e+02**/1.1e+03	**5.4e+03**/1.9e+04	**2.7e+02**/1.1e+03	5.8e+06/**1.4e+06**
sumMassInRange	F	**2.0e+08**/6.1e+08	**4.7e+08**/1.3e+09	**5.3e+06**/6.4e+06	**1.8e+02**/1.7e+03	3.3e+04/**2.6e+04**	**1.7e+02**/1.7e+03	5.6e+06/**1.6e+06**
twoPointCorrelation bin_8_16	F	**2.0e+08**/5.6e+08	**6.1e+08**/1.4e+09	4.9e+06/**3.7e+06**	**3.0e+02**/1.2e+03	**8.9e+03**/1.7e+04	**2.8e+02**/1.1e+03	5.8e+06/**1.3e+06**
Total		1.3e+09/3.6e+09	3.5e+09/8.4e+09	3.3e+07/3.2e+07	2.1e+03/1.1e+04	6.1e+05/1.3e+05	1.9e+03/1.0e+04	3.3e+07/8.9e+06

Table 25: Per-pass performance for `LinearListReduction`, ADT has 11 fields, SoA uses 11 buffers (mutable + immutable cursors). Times are median per iteration (s); ± shows standard error of the mean across –iterate runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup >1× means SoA is faster. OOM = out of memory.

Pass	**T**	**Uses**	**Dead%**	**Am**	**Ai**	**Sm**	**Si**	**Am/Sm**	**Ai/Am**	**Ai/Sm**	**Ai/Si**
reduction	F	2/11	82%	0.0306	0.1329	**0.0052**	*OOM*	**5.92**×	**4.34**×	**25.74**×	–
Total				0.0306	0.1329	**0.0052**	–	**5.92**×	**4.34**×	**25.74**×	–
Geomean								**5.92**×	**4.34**×	**25.74**×	–

Table 26: Per-pass PAPI counters for `LinearListReduction` (A/S). Each cell is median counter value pair per pass. Counter headers are abbreviated for compactness: CYC=CPU cycles, L1D/L1I/L2D/L2I=cache misses, LLC=LLC load misses.

Pass	**T**	**CYC (A/S)**	**INS (A/S)**	**L1D (A/S)**	**L1I (A/S)**	**L2D (A/S)**	**L2I (A/S)**	**LLC (A/S)**
reduction	F	1.4e+08/**2.6e+07**	**9.0e+07**/1.0e+08	1.5e+06/**4.7e+04**	7.9e+02/**2.7e+02**	3.3e+05/**7.1e+03**	6.4e+02/**1.6e+02**	1.2e+07/**1.2e+06**
Total		1.4e+08/2.6e+07	9.0e+07/1.0e+08	1.5e+06/4.7e+04	7.9e+02/2.7e+02	3.3e+05/7.1e+03	6.4e+02/1.6e+02	1.2e+07/1.2e+06

Table 27: Per-pass PAPI counters for `LinearListReduction` (Ai/Si). Each cell is median counter value pair per pass. Counter headers are abbreviated for compactness: CYC=CPU cycles, L1D/L1I/L2D/L2I=cache misses, LLC=LLC load misses.

Pass	**T**	**CYC (Ai/Si)**	**INS (Ai/Si)**	**L1D (Ai/Si)**	**L1I (Ai/Si)**	**L2D (Ai/Si)**	**L2I (Ai/Si)**	**LLC (Ai/Si)**
reduction	F	6.7e+08/–	2.5e+08/–	4.0e+06/–	8.9e+02/–	4.0e+05/–	7.4e+02/–	2.7e+07/–
Total		6.7e+08/–	2.5e+08/–	4.0e+06/–	8.9e+02/–	4.0e+05/–	7.4e+02/–	2.7e+07/–

Table 28: Per-pass performance for `List`, ADT has 2 fields, SoA uses 2 buffers (mutable + immutable cursors). Times are median per iteration (s); ± shows standard error of the mean across –iterate runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup >1× means SoA is faster. OOM = out of memory.

Pass	**T**	**Uses**	**Dead%**	**Am**	**Ai**	**Sm**	**Si**	**Am/Sm**	**Ai/Am**	**Ai/Sm**	**Ai/Si**
add1 List	M	2/2	0%	**0.2533**	*OOM*	0.3391	*OOM*	0.75×	–	–	–
length List	F	1/2	50%	0.0414	*OOM*	**0.0216**	*OOM*	**1.91**×	–	–	–
sumList	F	2/2	0%	**0.0440**	*OOM*	0.0453	*OOM*	0.97×	–	–	–
sumList tail recursive	F	2/2	0%	**0.0443**	*OOM*	0.0446	*OOM*	0.99×	–	–	–
Total				**0.3830**	–	0.4506	–	0.85×	–	–	–
Geomean								1.08×	–	–	–

Table 29: Per-pass PAPI counters for `List` (A/S). Each cell is median counter value pair per pass. Counter headers are abbreviated for compactness: CYC=CPU cycles, L1D/L1I/L2D/L2I=cache misses, LLC=LLC load misses.

Pass	**T**	**CYC (A/S)**	**INS (A/S)**	**L1D (A/S)**	**L1I (A/S)**	**L2D (A/S)**	**L2I (A/S)**	**LLC (A/S)**
add1 List	M	**5.9e+08**/9.7e+08	**1.9e+09**/2.8e+09	7.9e+06/**1.8e+06**	**2.6e+05**/5.5e+05	**1.7e+04**/6.4e+04	**9.3e+03**/1.0e+04	9.7e+06/**7.6e+06**
length List	F	2.1e+08/**1.1e+08**	8.0e+08/**8.0e+08**	1.2e+05/**8.4e+04**	2.7e+02/**8.3e+01**	**2.8e+03**/1.4e+04	2.5e+02/**7.2e+01**	1.3e+07/**1.2e+06**
sumList	F	**2.2e+08**/2.3e+08	**9.0e+08**/1.0e+09	6.7e+05/**5.3e+05**	1.4e+02/**8.4e+01**	**3.5e+03**/6.8e+04	1.2e+02/**6.4e+01**	1.3e+07/**1.2e+07**
sumList tail recursive	F	**2.2e+08**/2.3e+08	**9.0e+08**/1.0e+09	**5.0e+05**/5.6e+05	**1.6e+02**/2.0e+02	**3.3e+03**/5.9e+04	**1.2e+02**/1.9e+02	1.3e+07/**1.2e+07**
Total		1.2e+09/1.5e+09	4.5e+09/5.6e+09	9.2e+06/3.0e+06	2.6e+05/5.5e+05	2.6e+04/2.0e+05	9.8e+03/1.1e+04	4.9e+07/3.3e+07

Table 30: Per-pass PAPI counters for `List` (Ai/Si). Each cell is median counter value pair per pass. Counter headers are abbreviated for compactness: CYC=CPU cycles, L1D/L1I/L2D/L2I=cache misses, LLC=LLC load misses.

Pass	**T**	**CYC (Ai/Si)**	**INS (Ai/Si)**	**L1D (Ai/Si)**	**L1I (Ai/Si)**	**L2D (Ai/Si)**	**L2I (Ai/Si)**	**LLC (Ai/Si)**
Total		–/–	–/–	–/–	–/–	–/–	–/–	–/–

Table 31: Per-pass performance for `MonoTree`, ADT has 3 fields, SoA uses 2 buffers (mutable + immutable cursors). Times are median per iteration (s); ± shows standard error of the mean across –iterate runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup >1× means SoA is faster. OOM = out of memory.

Pass	T	Uses	Dead%	Am	Ai	Sm	Si	Am/Sm	Ai/Am	Ai/Sm	Ai/Si
add1Tree	M	3/3	0%	**0.1006**	0.1027	0.1086	0.1211	0.93×	1.02×	0.95×	0.85×
sumTree	F	3/3	0%	**0.0091**	0.0403	0.0113	0.0806	0.81×	**4.42**×	**3.58**×	0.50×
sumTree TailRec	F	3/3	0%	**0.0086**	0.0402	0.0112	0.0808	0.77×	**4.65**×	**3.60**×	0.50×
Total				**0.1184**	0.1833	0.1310	0.2825	0.90×	**1.55**×	**1.40**×	0.65×
Geomean								0.83×	**2.76**×	**2.30**×	0.60×

Table 32: Per-pass PAPI counters for `MonoTree` (A/S). Each cell is median counter value pair per pass. Counter headers are abbreviated for compactness: CYC=CPU cycles, L1D/L1I/L2D/L2I=cache misses, LLC=LLC load misses.

Pass	T	CYC (A/S)	INS (A/S)	L1D (A/S)	L1I (A/S)	L2D (A/S)	L2I (A/S)	LLC (A/S)
add1Tree	M	1.6e+08/**1.4e+08**	**5.0e+08**/5.9e+08	**2.0e+05**/4.7e+05	**3.4e+04**/6.6e+04	**2.4e+03**/5.2e+03	1.5e+03/**1.4e+03**	4.6e+05/**4.2e+05**
sumTree	F	**4.6e+07**/5.7e+07	**2.2e+08**/2.6e+08	**6.8e+03**/4.1e+04	**2.5e+02**/4.6e+02	**6.7e+02**/4.0e+03	**2.2e+02**/4.5e+02	8.5e+05/**6.7e+05**
sumTree TailRec	F	**4.4e+07**/5.6e+07	**1.9e+08**/2.2e+08	**6.3e+03**/2.1e+04	**2.4e+02**/4.5e+02	**7.6e+02**/4.0e+03	**2.1e+02**/4.3e+02	8.5e+05/**6.7e+05**
Total		2.5e+08/2.6e+08	9.1e+08/1.1e+09	2.2e+05/5.4e+05	3.4e+04/6.7e+04	3.8e+03/1.3e+04	2.0e+03/2.3e+03	2.2e+06/1.8e+06

Table 33: Per-pass PAPI counters for `MonoTree` (Ai/Si). Each cell is median counter value pair per pass. Counter headers are abbreviated for compactness: CYC=CPU cycles, L1D/L1I/L2D/L2I=cache misses, LLC=LLC load misses.

Pass	T	CYC (Ai/Si)	INS (Ai/Si)	L1D (Ai/Si)	L1I (Ai/Si)	L2D (Ai/Si)	L2I (Ai/Si)	LLC (Ai/Si)
add1Tree	M	**2.4e+08**/5.0e+08	**7.1e+08**/1.2e+09	**8.7e+05**/1.0e+06	**3.4e+04**/5.1e+04	**1.5e+04**/1.6e+04	**2.4e+03**/2.5e+03	**3.8e+05**/4.2e+05
sumTree	F	**2.0e+08**/4.1e+08	**4.3e+08**/6.3e+08	**6.6e+04**/9.4e+05	**2.6e+02**/2.7e+02	**9.7e+02**/1.5e+04	**2.2e+02**/2.3e+02	**3.5e+05**/3.9e+05
sumTree TailRec	F	**2.0e+08**/4.1e+08	**4.1e+08**/6.4e+08	**6.0e+04**/9.2e+05	2.9e+02/**2.3e+02**	**9.4e+02**/1.4e+04	2.6e+02/**1.9e+02**	**3.6e+05**/4.0e+05
Total		6.5e+08/1.3e+09	1.6e+09/2.5e+09	1.0e+06/2.9e+06	3.5e+04/5.2e+04	1.6e+04/4.4e+04	2.9e+03/3.0e+03	1.1e+06/1.2e+06

Table 34: Per-pass performance for `ObjectGraph`, ADT has 5 fields, SoA uses 4 buffers (mutable + immutable cursors). Times are median per iteration (s); ± shows standard error of the mean across –iterate runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup >1× means SoA is faster. OOM = out of memory.

Pass	T	Uses	Dead%	Am	Ai	Sm	Si	Am/Sm	Ai/Am	Ai/Sm	Ai/Si
countLargeItems	F	3/5	40%	**0.0146**	0.0445	0.0651	0.0464	0.22×	**3.06**×	0.68×	0.96×
countMarked	F	3/5	40%	**0.0188**	0.0445	0.0207	0.0471	0.91×	**2.36**×	**2.15**×	0.94×
countSurvivors	F	4/5	20%	**0.0141**	0.0442	0.0144	0.0468	0.98×	**3.13**×	**3.08**×	0.95×
deadBytes	F	4/5	20%	**0.0139**	0.0440	0.0292	0.0472	0.48×	**3.16**×	**1.51**×	0.93×
liveBytes	F	4/5	20%	**0.0142**	0.0438	0.0710	0.0468	0.20×	**3.08**×	0.62×	0.94×
sumObjIds	F	3/5	40%	0.0130	0.0432	**0.0107**	0.0476	**1.21**×	**3.32**×	**4.02**×	0.91×
sweepUnmarked	M	5/5	0%	**0.0713**	0.0878	0.0801	0.1870	0.89×	**1.23**×	1.10×	0.47×
totalHeapSize	F	3/5	40%	0.0256	0.0645	**0.0203**	0.0506	**1.26**×	**2.52**×	**3.18**×	**1.28**×
touchHotObjects	M	5/5	0%	**0.0694**	0.0869	0.0804	0.1930	0.86×	**1.25**×	1.08×	0.45×
Total				**0.2550**	0.5034	0.3919	0.7123	0.65×	**1.97**×	**1.28**×	0.71×
Geomean								0.66×	**2.42**×	**1.59**×	0.83×

Table 35: Per-pass PAPI counters for `ObjectGraph` (A/S). Each cell is median counter value pair per pass. Counter headers are abbreviated for compactness: CYC=CPU cycles, L1D/L1I/L2D/L2I=cache misses, LLC=LLC load misses.

Pass	T	CYC (A/S)	INS (A/S)	L1D (A/S)	L1I (A/S)	L2D (A/S)	L2I (A/S)	LLC (A/S)
countLargeItems	F	6.6e+07/**5.2e+07**	**2.6e+08**/2.8e+08	1.1e+06/**4.0e+05**	**3.8e+02**/8.8e+02	**6.1e+03**/7.2e+03	**3.5e+02**/7.4e+02	2.8e+06/**4.5e+05**
countMarked	F	**5.5e+07**/5.6e+07	**2.6e+08**/2.8e+08	1.2e+06/**7.1e+05**	**4.1e+02**/1.0e+03	**3.0e+03**/1.4e+04	**3.8e+02**/9.7e+02	2.5e+06/**5.6e+05**
countSurvivors	F	7.1e+07/**6.5e+07**	**3.1e+08**/3.8e+08	9.7e+05/**1.5e+05**	**3.0e+02**/5.7e+02	**2.5e+03**/5.6e+03	**2.9e+02**/5.4e+02	2.8e+06/**1.2e+06**
deadBytes	F	7.0e+07/**6.1e+07**	**2.7e+08**/3.5e+08	7.7e+05/**1.1e+05**	**3.8e+02**/4.9e+02	**2.6e+03**/5.6e+03	**3.5e+02**/4.4e+02	2.8e+06/**9.5e+05**
liveBytes	F	7.2e+07/**6.0e+07**	**2.7e+08**/3.5e+08	6.8e+05/**1.2e+05**	**3.3e+02**/1.3e+03	**2.1e+03**/5.6e+03	**3.0e+02**/1.2e+03	2.8e+06/**7.7e+05**
sumObjIds	F	6.6e+07/**5.4e+07**	**2.5e+08**/2.7e+08	1.3e+06/**7.1e+04**	**2.4e+02**/4.4e+02	**3.1e+03**/4.5e+03	**2.2e+02**/4.1e+02	2.8e+06/**6.9e+05**
sweepUnmarked	M	**1.9e+08**/2.3e+08	**5.5e+08**/8.8e+08	1.1e+06/**4.2e+05**	**7.2e+04**/1.4e+05	**6.9e+03**/1.1e+04	**6.1e+03**/9.9e+03	2.0e+06/**1.0e+06**
totalHeapSize	F	**4.8e+07**/5.4e+07	**2.5e+08**/2.7e+08	1.6e+06/**1.0e+06**	**5.9e+02**/7.2e+02	**6.3e+03**/2.0e+04	**5.0e+02**/6.3e+02	2.2e+06/**5.1e+05**
touchHotObjects	M	**1.8e+08**/2.3e+08	**5.7e+08**/9.0e+08	1.0e+06/**4.9e+05**	**7.4e+04**/1.1e+05	**3.6e+03**/1.1e+04	**3.4e+03**/7.8e+03	1.8e+06/**1.0e+06**
Total		8.1e+08/8.7e+08	3.0e+09/4.0e+09	9.8e+06/3.5e+06	1.5e+05/2.5e+05	3.6e+04/8.4e+04	1.2e+04/2.3e+04	2.2e+07/7.1e+06

Table 36: Per-pass PAPI counters for `ObjectGraph` (Ai/Si). Each cell is median counter value pair per pass. Counter headers are abbreviated for compactness: CYC=CPU cycles, L1D/L1I/L2D/L2I=cache misses, LLC=LLC load misses.

Pass	T	CYC (Ai/Si)	INS (Ai/Si)	L1D (Ai/Si)	L1I (Ai/Si)	L2D (Ai/Si)	L2I (Ai/Si)	LLC (Ai/Si)
countLargeItems	F	**2.3e+08**/2.4e+08	**4.6e+08**/7.9e+08	5.6e+05/**4.4e+05**	3.7e+02/**2.0e+02**	**4.8e+03**/8.3e+03	3.3e+02/**1.9e+02**	1.8e+06/**4.3e+05**
countMarked	F	**2.3e+08**/2.4e+08	**4.6e+08**/7.9e+08	5.5e+05/**4.9e+05**	**1.9e+02**/5.0e+02	**5.3e+03**/1.0e+04	**1.6e+02**/4.6e+02	1.8e+06/**4.5e+05**
countSurvivors	F	**2.2e+08**/2.4e+08	**5.5e+08**/8.2e+08	**2.0e+05**/3.7e+05	**1.4e+02**/1.5e+02	**2.0e+03**/8.0e+03	**1.2e+02**/1.3e+02	1.8e+06/**7.4e+05**
deadBytes	F	**2.2e+08**/2.4e+08	**4.9e+08**/8.1e+08	**3.7e+05**/3.8e+05	**3.0e+02**/4.2e+02	**3.7e+03**/8.1e+03	**2.6e+02**/3.8e+02	1.8e+06/**7.4e+05**
liveBytes	F	**2.2e+08**/2.4e+08	**5.0e+08**/8.1e+08	**1.4e+05**/3.7e+05	2.9e+02/**2.2e+02**	**1.9e+03**/7.7e+03	2.5e+02/**2.0e+02**	1.8e+06/**7.4e+05**
sumObjIds	F	**2.2e+08**/2.4e+08	**4.4e+08**/7.8e+08	5.4e+05/**4.9e+05**	**3.1e+02**/4.2e+02	**4.6e+03**/1.0e+04	**2.8e+02**/3.8e+02	1.8e+06/**4.2e+05**
sweepUnmarked	M	**2.7e+08**/7.7e+08	**7.6e+08**/2.1e+09	**1.8e+05**/2.4e+06	**7.4e+04**/1.3e+05	**3.8e+03**/1.8e+04	**4.1e+03**/6.6e+03	1.8e+06/**9.5e+05**
totalHeapSize	F	**2.3e+08**/2.4e+08	**4.4e+08**/7.8e+08	1.9e+06/**5.7e+05**	**5.6e+02**/6.4e+02	3.0e+04/**1.1e+04**	**4.2e+02**/4.7e+02	1.4e+06/**4.2e+05**
touchHotObjects	M	**2.7e+08**/8.0e+08	**7.7e+08**/2.2e+09	**2.3e+05**/1.8e+06	**9.8e+04**/1.6e+05	**3.2e+03**/1.8e+04	**3.8e+03**/5.2e+03	1.7e+06/**9.4e+05**
Total		2.1e+09/3.2e+09	4.9e+09/9.9e+09	4.6e+06/7.3e+06	1.7e+05/3.0e+05	5.9e+04/1.0e+05	9.6e+03/1.4e+04	1.6e+07/5.8e+06

Table 37: Per-pass performance for `PiecewiseFunctions`, ADT has 8 fields, SoA uses 7 buffers (mutable + immutable cursors). Times are median per iteration (s); ± shows standard error of the mean across –iterate runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup >1× means SoA is faster. OOM = out of memory.

Pass	T	Uses	Dead%	Am	Ai	Sm	Si	Am/Sm	Ai/Am	Ai/Sm	Ai/Si
addConstPW	M	8/8	0%	**0.1198**	0.1309	0.1335	0.4021	0.90×	1.09×	0.98×	0.33×
autorefineMaxLevel	F	4/8	50%	**0.0206**	0.0446	0.0212	0.2363	0.97×	**2.16**×	**2.10**×	0.19×
compressMass	F	3/8	62%	0.0195	0.0450	**0.0115**	0.2355	**1.69**×	**2.30**×	**3.91**×	0.19×
diffPW	M	8/8	0%	**0.1196**	0.1322	0.1307	0.4061	0.92×	**1.10**×	1.01×	0.33×
lbDeuxLoadProxy	F	5/8	38%	0.0215	0.0470	**0.0186**	0.2387	**1.16**×	**2.19**×	**2.52**×	0.20×
norm2Estimate	F	5/8	38%	0.0318	0.0752	**0.0245**	0.1987	**1.30**×	**2.36**×	**3.07**×	0.38×
pmapCutHistogram	F	4/8	50%	0.0321	0.0557	**0.0124**	0.2475	**2.58**×	**1.73**×	**4.48**×	0.23×
truncateTolViolations	F	3/8	62%	0.0200	0.0510	**0.0115**	0.2112	**1.74**×	**2.54**×	**4.43**×	0.24×
Total				0.3851	0.5816	**0.3640**	2.176	1.06×	**1.51**×	**1.60**×	0.27×
Geomean								**1.32**×	**1.85**×	**2.44**×	0.25×

Table 38: Per-pass PAPI counters for `PiecewiseFunctions` (A/S). Each cell is median counter value pair per pass. Counter headers are abbreviated for compactness: CYC=CPU cycles, L1D/L1I/L2D/L2I=cache misses, LLC=LLC load misses.

Pass	T	CYC (A/S)	INS (A/S)	L1D (A/S)	L1I (A/S)	L2D (A/S)	L2I (A/S)	LLC (A/S)
addConstPW	M	**2.8e+08**/3.5e+08	**5.7e+08**/1.2e+09	**1.3e+06**/1.8e+06	2.8e+05/**2.0e+05**	**1.4e+04**/2.2e+04	**3.9e+03**/2.0e+04	4.5e+06/**1.7e+06**
autorefineMaxLevel	F	**1.0e+08**/1.1e+08	**2.5e+08**/3.7e+08	2.5e+06/**3.0e+05**	6.6e+02/**2.0e+02**	**5.4e+03**/7.0e+03	6.6e+02/**1.7e+02**	6.2e+06/**8.7e+05**
compressMass	F	9.9e+07/**5.8e+07**	**2.5e+08**/2.8e+08	2.2e+06/**3.2e+04**	6.8e+02/**3.4e+02**	2.0e+04/**5.3e+03**	6.7e+02/**3.0e+02**	6.3e+06/**6.9e+05**
diffPW	M	**2.8e+08**/3.4e+08	**5.9e+08**/1.3e+09	**8.5e+05**/2.2e+06	**1.4e+05**/2.1e+05	**8.9e+03**/2.5e+04	**7.1e+03**/2.1e+04	4.1e+06/**1.8e+06**
lbDeuxLoadProxy	F	1.1e+08/**9.5e+07**	**3.3e+08**/4.9e+08	2.4e+06/**4.4e+05**	5.0e+02/**1.5e+02**	1.4e+04/**9.0e+03**	5.0e+02/**1.3e+02**	6.3e+06/**1.5e+06**
norm2Estimate	F	1.1e+08/**9.6e+07**	**3.4e+08**/4.4e+08	**1.2e+06**/2.3e+06	5.2e+02/**4.6e+02**	**1.1e+04**/4.2e+04	4.4e+02/**3.8e+02**	5.4e+06/**1.0e+06**
pmapCutHistogram	F	1.6e+08/**6.3e+07**	**2.8e+08**/3.7e+08	1.5e+06/**6.2e+04**	7.2e+02/**4.9e+02**	9.1e+04/**7.3e+03**	7.1e+02/**4.6e+02**	4.8e+06/**1.3e+06**
truncateTolViolations	F	1.0e+08/**5.8e+07**	**2.5e+08**/2.9e+08	2.1e+06/**4.9e+04**	7.0e+02/**3.5e+02**	1.6e+04/**4.9e+03**	6.8e+02/**3.1e+02**	6.3e+06/**7.0e+05**
Total		1.2e+09/1.2e+09	2.8e+09/4.7e+09	1.4e+07/7.2e+06	4.2e+05/4.2e+05	1.8e+05/1.2e+05	1.5e+04/4.2e+04	4.4e+07/9.5e+06

Table 39: Per-pass PAPI counters for `PiecewiseFunctions` (Ai/Si). Each cell is median counter value pair per pass. Counter headers are abbreviated for compactness: CYC=CPU cycles, L1D/L1I/L2D/L2I=cache misses, LLC=LLC load misses.

Pass	T	CYC (Ai/Si)	INS (Ai/Si)	L1D (Ai/Si)	L1I (Ai/Si)	L2D (Ai/Si)	L2I (Ai/Si)	LLC (Ai/Si)
addConstPW	M	**3.3e+08**/1.7e+09	**7.8e+08**/3.3e+09	**3.0e+05**/5.0e+06	**1.4e+05**/2.5e+05	**7.1e+03**/3.9e+04	**5.0e+03**/3.2e+04	4.2e+06/**1.9e+06**
autorefineMaxLevel	F	**2.3e+08**/1.2e+09	**4.9e+08**/1.3e+09	**2.9e+05**/2.1e+06	**2.0e+02**/1.7e+03	**2.3e+03**/4.1e+04	**1.8e+02**/7.3e+02	4.2e+06/**7.2e+05**
compressMass	F	**2.3e+08**/1.2e+09	**4.7e+08**/1.2e+09	**2.8e+05**/1.3e+06	**2.6e+02**/1.0e+03	**2.0e+03**/3.0e+04	2.3e+02/**2.2e+02**	4.1e+06/**4.1e+05**
diffPW	M	**3.4e+08**/1.7e+09	**8.1e+08**/3.4e+09	**3.1e+05**/5.1e+06	**1.3e+05**/3.1e+05	**5.0e+03**/4.8e+04	**5.7e+03**/3.1e+04	4.1e+06/**1.9e+06**
lbDeuxLoadProxy	F	**2.4e+08**/1.2e+09	**5.3e+08**/1.3e+09	**1.6e+05**/2.6e+06	**4.2e+02**/2.1e+03	**2.0e+03**/4.2e+04	**3.9e+02**/1.5e+03	4.2e+06/**1.0e+06**
norm2Estimate	F	**3.5e+08**/1.0e+09	**5.1e+08**/1.3e+09	6.4e+06/**3.5e+06**	**5.9e+02**/1.6e+03	1.3e+05/**6.7e+04**	**4.7e+02**/1.3e+03	3.6e+06/**9.1e+05**
pmapCutHistogram	F	**2.8e+08**/1.3e+09	**5.0e+08**/1.3e+09	**7.4e+05**/2.0e+06	**4.8e+02**/5.0e+02	**1.1e+04**/3.8e+04	4.4e+02/**3.1e+02**	4.0e+06/**7.2e+05**
truncateTolViolations	F	**2.6e+08**/1.1e+09	**4.4e+08**/1.3e+09	2.6e+06/**1.4e+06**	**6.8e+02**/1.3e+03	5.5e+04/**3.0e+04**	6.5e+02/**2.8e+02**	4.0e+06/**3.8e+05**
Total		2.3e+09/1.0e+10	4.5e+09/1.4e+10	1.1e+07/2.3e+07	2.7e+05/5.7e+05	2.1e+05/3.3e+05	1.3e+04/6.7e+04	3.2e+07/7.9e+06

Table 40: Per-pass performance for `TernaryTree`, ADT has 5 fields, SoA uses 3 buffers (mutable + immutable cursors). Times are median per iteration (s); ± shows standard error of the mean across –iterate runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup >1× means SoA is faster. OOM = out of memory.

Pass	T	Uses	Dead%	Am	Ai	Sm	Si	Am/Sm	Ai/Am	Ai/Sm	Ai/Si
add 1 tree	M	5/5	0%	0.1028	0.1126	**0.0916**	0.3449	**1.12**×	1.10×	**1.23**×	0.33×
sum tree	F	5/5	0%	0.0272	0.0501	**0.0248**	0.2733	1.10×	**1.84**×	**2.02**×	0.18×
Total				0.1300	0.1627	**0.1164**	0.6182	**1.12**×	**1.25**×	**1.40**×	0.26×
Geomean								**1.11**×	**1.42**×	**1.58**×	0.24×

Table 41: Per-pass PAPI counters for `TernaryTree` (A/S). Each cell is median counter value pair per pass. Counter headers are abbreviated for compactness: CYC=CPU cycles, L1D/L1I/L2D/L2I=cache misses, LLC=LLC load misses.

Pass	T	CYC (A/S)	INS (A/S)	L1D (A/S)	L1I (A/S)	L2D (A/S)	L2I (A/S)	LLC (A/S)
add 1 tree	M	2.8e+08/**2.3e+08**	**7.5e+08**/9.8e+08	1.9e+06/**9.8e+05**	**6.8e+04**/1.2e+05	3.2e+04/**1.1e+04**	3.6e+03/**2.0e+03**	1.4e+06/**1.1e+06**
sum tree	F	1.4e+08/**1.3e+08**	**4.6e+08**/5.4e+08	**8.7e+04**/3.6e+05	**1.2e+02**/1.4e+02	**8.9e+02**/9.1e+03	**1.1e+02**/1.2e+02	1.8e+06/**1.3e+06**
Total		4.2e+08/3.5e+08	1.2e+09/1.5e+09	2.0e+06/1.3e+06	6.8e+04/1.2e+05	3.3e+04/2.0e+04	3.7e+03/2.1e+03	3.2e+06/2.4e+06

Table 42: Per-pass PAPI counters for `TernaryTree` (Ai/Si). Each cell is median counter value pair per pass. Counter headers are abbreviated for compactness: CYC=CPU cycles, L1D/L1I/L2D/L2I=cache misses, LLC=LLC load misses.

Pass	T	CYC (Ai/Si)	INS (Ai/Si)	L1D (Ai/Si)	L1I (Ai/Si)	L2D (Ai/Si)	L2I (Ai/Si)	LLC (Ai/Si)
add 1 tree	M	**3.3e+08**/1.6e+09	**9.5e+08**/2.4e+09	**8.9e+05**/2.3e+06	**6.7e+04**/1.8e+05	**1.5e+04**/4.5e+04	**3.4e+03**/4.1e+03	1.4e+06/**9.6e+05**
sum tree	F	**2.5e+08**/1.4e+09	**5.8e+08**/1.0e+09	**8.2e+04**/2.3e+06	**1.4e+02**/1.2e+03	**1.4e+03**/4.1e+04	**1.3e+02**/4.8e+02	1.4e+06/**9.4e+05**
Total		5.8e+08/3.0e+09	1.5e+09/3.4e+09	9.7e+05/4.6e+06	6.8e+04/1.8e+05	1.6e+04/8.6e+04	3.5e+03/4.6e+03	2.8e+06/1.9e+06

Table 43: Per-pass performance for `Trie`, ADT has 10 fields, SoA uses 9 buffers (mutable + immutable cursors). Times are median per iteration (s); ± shows standard error of the mean across –iterate runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup >1× means SoA is faster. OOM = out of memory.

Pass	T	Uses	Dead%	Am	Ai	Sm	Si	Am/Sm	Ai/Am	Ai/Sm	Ai/Si
autocompleteTopKProxy	F	5/10	50%	0.0139	0.0273	**0.0093**	0.1077	**1.49**×	**1.96**×	**2.93**×	0.25×
countLazyNodes	F	4/10	60%	0.0123	0.0276	**0.0066**	0.1109	**1.86**×	**2.24**×	**4.16**×	0.25×
countTerminals	F	3/10	70%	0.0153	0.0256	**0.0098**	0.0988	**1.56**×	**1.68**×	**2.62**×	0.26×
decayTrieStats	M	10/10	0%	**0.0740**	0.0793	0.0861	0.2373	0.86×	1.07×	0.92×	0.33×
resetTraversalState	M	10/10	0%	**0.0746**	0.2037	0.0814	0.2461	0.92×	**2.73**×	**2.50**×	0.83×
sumPrefixFreq	F	3/10	70%	0.0201	0.0412	**0.0118**	0.1091	**1.70**×	**2.05**×	**3.49**×	0.38×
sumSubtreeHints	F	3/10	70%	0.0130	0.0260	**0.0077**	0.1055	**1.69**×	**2.00**×	**3.38**×	0.25×
Total				0.2232	0.4307	**0.2127**	1.016	1.05×	**1.93**×	**2.03**×	0.42×
Geomean								**1.39**×	**1.90**×	**2.63**×	0.33×

Table 44: Per-pass PAPI counters for `Trie` (A/S). Each cell is median counter value pair per pass. Counter headers are abbreviated for compactness: CYC=CPU cycles, L1D/L1I/L2D/L2I=cache misses, LLC=LLC load misses.

Pass	T	CYC (A/S)	INS (A/S)	L1D (A/S)	L1I (A/S)	L2D (A/S)	L2I (A/S)	LLC (A/S)
autocompleteTopKProxy	F	7.0e+07/**4.2e+07**	**1.3e+08**/2.1e+08	1.5e+06/**1.7e+05**	1.7e+02/**1.2e+02**	4.5e+04/**4.6e+03**	1.5e+02/**1.1e+02**	3.9e+06/**8.0e+05**
countLazyNodes	F	6.3e+07/**3.3e+07**	**1.3e+08**/1.6e+08	1.3e+06/**2.2e+04**	**1.5e+02**/1.6e+02	1.7e+04/**3.3e+03**	**1.3e+02**/1.4e+02	4.2e+06/**6.6e+05**
countTerminals	F	4.3e+07/**3.0e+07**	**1.0e+08**/1.2e+08	1.8e+06/**2.4e+05**	2.3e+02/**1.8e+02**	4.1e+04/**6.0e+03**	2.1e+02/**1.7e+02**	3.8e+06/**2.8e+05**
decayTrieStats	M	**1.6e+08**/2.1e+08	**3.5e+08**/7.9e+08	**6.9e+05**/1.1e+06	**8.7e+04**/1.1e+05	**6.2e+03**/2.0e+04	**3.4e+03**/2.1e+04	3.0e+06/**1.2e+06**
resetTraversalState	M	**1.6e+08**/1.9e+08	**2.8e+08**/7.0e+08	9.8e+05/**7.6e+05**	**8.7e+04**/1.6e+05	**8.1e+03**/1.4e+04	**2.8e+03**/1.3e+04	3.3e+06/**9.0e+05**
sumPrefixFreq	F	4.0e+07/**3.0e+07**	**1.1e+08**/1.2e+08	2.0e+06/**6.4e+05**	6.4e+02/**4.2e+02**	4.1e+04/**1.3e+04**	4.6e+02/**2.8e+02**	3.5e+06/**2.4e+05**
sumSubtreeHints	F	5.3e+07/**2.8e+07**	**1.1e+08**/1.2e+08	1.7e+06/**1.3e+04**	2.2e+02/**1.3e+02**	3.3e+04/**2.4e+03**	2.0e+02/**1.1e+02**	4.0e+06/**3.3e+05**
Total		5.9e+08/5.7e+08	1.2e+09/2.2e+09	9.9e+06/3.0e+06	1.8e+05/2.7e+05	1.9e+05/6.3e+04	7.3e+03/3.5e+04	2.6e+07/4.4e+06

Table 45: Per-pass PAPI counters for `Trie` (Ai/Si). Each cell is median counter value pair per pass. Counter headers are abbreviated for compactness: CYC=CPU cycles, L1D/L1I/L2D/L2I=cache misses, LLC=LLC load misses.

Pass	**T**	**CYC (Ai/Si)**	**INS (Ai/Si)**	**L1D (Ai/Si)**	**L1I (Ai/Si)**	**L2D (Ai/Si)**	**L2I (Ai/Si)**	**LLC (Ai/Si)**
autocompleteTopKProxy	F	**1.4e+08**/5.5e+08	**2.3e+08**/7.5e+08	**1.1e+06**/1.5e+06	**1.6e+02**/1.3e+03	**4.5e+03**/2.7e+04	**1.5e+02**/1.0e+03	2.8e+06/**4.9e+05**
countLazyNodes	F	**1.4e+08**/5.6e+08	**2.2e+08**/7.6e+08	2.2e+06/**1.1e+06**	**1.8e+02**/1.3e+03	**4.1e+03**/2.2e+04	**1.6e+02**/5.0e+02	2.7e+06/**3.5e+05**
countTerminals	F	**1.3e+08**/5.0e+08	**2.0e+08**/7.3e+08	1.2e+06/**6.5e+05**	**1.7e+02**/6.0e+02	**2.3e+03**/1.6e+04	**1.5e+02**/2.8e+02	2.9e+06/**2.0e+05**
decayTrieStats	M	**1.9e+08**/9.9e+08	**4.5e+08**/2.1e+09	**2.9e+05**/4.5e+06	**8.7e+04**/1.6e+05	**3.4e+03**/2.5e+04	**3.3e+03**/4.1e+04	2.9e+06/**1.2e+06**
resetTraversalState	M	**1.9e+08**/1.0e+09	**3.7e+08**/2.0e+09	**2.0e+06**/3.2e+06	**1.2e+05**/2.3e+05	5.9e+04/**2.6e+04**	**6.1e+03**/2.8e+04	2.4e+06/**9.6e+05**
sumPrefixFreq	F	**1.2e+08**/5.4e+08	**2.1e+08**/7.3e+08	1.7e+06/**7.0e+05**	**5.0e+02**/9.4e+02	**1.0e+04**/1.7e+04	**3.6e+02**/3.6e+02	2.5e+06/**1.9e+05**
sumSubtreeHints	F	**1.3e+08**/5.4e+08	**2.1e+08**/7.3e+08	1.4e+06/**6.1e+05**	**2.0e+02**/9.0e+02	**3.2e+03**/1.6e+04	**1.8e+02**/2.3e+02	2.9e+06/**2.1e+05**
Total		1.0e+09/4.7e+09	1.9e+09/7.8e+09	9.8e+06/1.2e+07	2.0e+05/3.9e+05	8.7e+04/1.5e+05	1.0e+04/7.1e+04	1.9e+07/3.6e+06

Table 46: Per-pass GHC comparison for `OctTree`. Times are median per iteration (s); ± shows standard error. GHC/Am and GHC/Sm are speedups.

Pass	**T**	**GHC**	**GHC/Am**	**GHC/Sm**
barnesHutPotential	F	5.207	**120.39**×	**117.00**×
clearFlags	M	5.299	**34.33**×	**30.74**×
countActive	F	5.340	**141.91**×	**177.03**×
countParticles	F	5.278	**146.66**×	**181.08**×
fmmPotential	F	6.092	**56.20**×	**59.69**×
scaleEnergy	M	5.827	**38.80**×	**33.10**×
sumEnergy	F	5.967	**132.30**×	**169.70**×
sumMass	F	3.220	**77.63**×	**105.36**×
paletteEntriesQuantized	F	7.179	**164.55**×	**188.03**×
quantizationErrorProxy	F	4.79e-04	0.01×	0.01×
Total		49.41	**70.02**×	**70.48**×
Geomean		–	**35.66**×	**39.19**×

Table 47: Per-pass GHC comparison for `Compiler`. Times are median per iteration (s); ± shows standard error. GHC/Am and GHC/Sm are speedups.

Pass	**T**	**GHC**	**GHC/Am**	**GHC/Sm**
blockCountPass	F	0.0458	**3.49**×	**12.62**×
branchStatsPass	F	0.0409	**3.46**×	**8.92**×
castInstCountPass	F	0.0420	**2.34**×	**20.20**×
has cycle	F	0.0359	1.04×	1.10×
instCountPass	F	0.6473	**42.13**×	**156.10**×
latencyModelPass	F	0.0935	**7.77**×	**27.09**×
memoryOpStatsPass	F	0.0424	**3.60**×	**7.58**×
stripSideEffectsPass	M	0.3139	**3.93**×	**3.46**×
targetReturnPass	M	0.2211	**2.78**×	**2.46**×
throughputModelPass	F	0.0928	**7.73**×	**31.35**×
verifyIR	F	0.4308	**21.19**×	**65.76**×
Total		2.007	**6.50**×	**8.15**×
Geomean		–	**5.11**×	**12.72**×

Table 48: Per-pass GHC comparison for `DBQuery`. Times are median per iteration (s); ± shows standard error. GHC/Am and GHC/Sm are speedups.

Pass	**T**	**GHC**	**GHC/Am**	**GHC/Sm**
clearQueryFlags	M	0.5913	**8.64**×	**7.05**×
countJoins	F	0.0967	**4.92**×	**8.63**×
filterSelectivitySkew	F	0.1329	**6.75**×	**10.81**×
hashJoinPressure	F	0.1048	**5.11**×	**8.36**×
scaleCosts	M	0.7463	**10.83**×	**8.85**×
sumCost	F	0.5334	**16.74**×	**25.39**×
sumMemory	F	0.1191	**6.00**×	**8.30**×
sumRows	F	0.1148	**4.00**×	**5.05**×
Total		2.439	**8.79**×	**9.30**×
Geomean		–	**7.09**×	**9.21**×

Table 49: Per-pass GHC comparison for `DecisionTree`. Times are median per iteration (s); ± shows standard error. GHC/Am and GHC/Sm are speedups.

Pass	**T**	**GHC**	**GHC/Am**	**GHC/Sm**
classify Batch	F	7.70e-05	**4.05**×	**1.22**×
classify Depth	F	1.00e-06	–	–
classify tree	F	4.00e-06	–	**2.00**×
countFeatureUses	F	0.1767	**2.92**×	**2.37**×
countLeaves	F	0.1892	**3.27**×	**2.80**×
countNodes	F	2.767	**46.55**×	**40.38**×
countSmallLeaves	F	0.1575	**2.60**×	**2.16**×
inferenceCost	F	0.1627	**2.80**×	**2.39**×
sumImpurity	F	0.1569	**2.64**×	**2.13**×
sumPathLengths	F	0.1764	**2.95**×	**2.27**×
sumSamples	F	0.1591	**2.69**×	**2.12**×
treeDepth	F	0.1612	**2.74**×	**2.37**×
Total		4.107	**7.70**×	**6.35**×
Geomean		–	**3.87**×	**2.80**×

Table 50: Per-pass GHC comparison for `DomTree`. Times are median per iteration (s); ± shows standard error. GHC/Am and GHC/Sm are speedups.

Pass	**T**	**GHC**	**GHC/Am**	**GHC/Sm**
SumArea	F	0.7972	**22.26**×	**39.20**×
computeWidths	M	0.1092	**3.67**×	**2.43**×
count styled	F	0.0519	**1.54**×	**4.43**×
find max Bottom	F	0.0677	**1.99**×	**3.59**×
scaleLayout	M	0.0883	**3.05**×	**2.38**×
sumTextWidth	F	0.0752	**2.23**×	**6.33**×
Total		1.189	**6.07**×	**8.21**×
Geomean		–	**3.46**×	**5.33**×

Table 51: Per-pass GHC comparison for `KDTree`. Times are median per iteration (s); ± shows standard error. GHC/Am and GHC/Sm are speedups.

Pass	**T**	**GHC**	**GHC/Am**	**GHC/Sm**
Find nearest Neighbour	F	3.70e-05	0.00×	0.00×
countInRange tight_box	F	5.00e-06	0.00×	0.00×
photonMappingBenchmark	F	0.9320	**22.32**×	**22.88**×
pointCloudNeighborhood	F	0.3029	**12.40**×	**18.35**×
sumMassInRange	F	4.00e-06	0.00×	0.00×
twoPointCorrelation bin_8_16	F	0.4605	**16.12**×	**22.25**×
Total		1.695	**9.41**×	**12.00**×
Geomean		–	0.07×	0.10×

Table 52: Per-pass GHC comparison for `LinearListReduction`. Times are median per iteration (s); ± shows standard error. GHC/Am and GHC/Sm are speedups.

Pass	**T**	**GHC**	**GHC/Am**	**GHC/Sm**
reduction	F	1.261	**41.23**×	**244.26**×
Total		1.261	**41.23**×	**244.26**×
Geomean		–	**41.23**×	**244.26**×